

Curso: APIs HTTP em Rust com axum

Oct 8, 2026 · @Daniel Freire

Como usar este curso

Você vai sair daqui sabendo construir, em Rust 1.99 com axum 0.8, uma API HTTP de produção: contrato em OpenAPI, erros padronizados, validação, segurança, testes de integração reais e observabilidade, organizada para continuar legível com centenas de rotas.

É a terceira versão do curso de APIs da série, depois de Minimal APIs no ASP.NET Core 10 e de APIs HTTP em Go: os mesmos onze módulos, o mesmo método **top-down** e a mesma API de cobrança como projeto final, com o mesmo contrato de erros. Rust fica num ponto curioso entre as outras duas: como no .NET, o framework extrai e converte a entrada a partir dos **tipos** dos parâmetros do handler; como no Go, não há contêiner de injeção de dependência nem convenções mágicas. E, ao contrário das duas, quase tudo isso é verificado em tempo de compilação: um handler com um parâmetro que o axum não sabe extrair simplesmente não compila.

Pré-requisitos

Rust com `async / await`, traits, `Result` e o operador `?`. O Módulo 3 do curso de processamento de arquivos em Rust (ownership, empréstimos e o runtime do tokio) é a melhor preparação; os idiomas de erro (`thiserror` nas bibliotecas, `anyhow` nas bordas) também vêm de lá.

Exemplo condutor

A mesma **API de cobrança**: recebe uploads de arquivos de retorno, informa o status de cada arquivo, consulta títulos e ocorrências e transmite o progresso do processamento em tempo real. Ela usa as mesmas tabelas que os workers de arquivos (em .NET, Go ou Rust) e devolve os mesmos `ProblemDetails` das outras duas APIs: um cliente que integra com qualquer uma delas trata erros com o mesmo código.

Stack usada

- Rust 1.99 (edição 2024), `tokio`
- `axum` 0.8 sobre `hyper`, com middlewares do ecossistema `tower` e `tower-http`
- `serde` e `serde_json` para JSON

- PostgreSQL com `sqlx`, que verifica as consultas contra o schema em tempo de compilação
- `utoipa` para gerar o documento OpenAPI a partir dos tipos
- `jsonwebtoken` para validar JWT, `governor` para rate limiting
- `tower::ServiceExt`, `testcontainers` e `proptest` para testes
- `tracing` e OpenTelemetry para observabilidade

A mesma API em três plataformas

Conceito	ASP.NET Core 10	Go 1.27	Rust com axum 0.8
Rotas	<code>MapGet</code> , <code>MapGroup</code>	<code>ServeMux</code> com <code>GET /v1/arquivos/{id}</code>	<code>Router::new().route("/v1/arquivos/{id}", get(h)) + nest</code>
Middleware	<code>app.Use(...)</code>	<code>func(http.Handler) http.Handler</code>	Camadas <code>tower (.layer(...))</code> e <code>middleware::from_fn</code>
Binding	Inferido pelos parâmetros	Explícito no handler	Extractors: inferido pelos tipos (<code>Path</code> , <code>Query</code> , <code>Json</code> , <code>State</code>)
Respostas tipadas	<code>TypedResults</code> , <code>Results<...></code>	Helpers + tipos de erro	<code>impl IntoResponse</code> , <code>Result<Json<T>, ErroDaApi></code>
Erros	Exceções + <code>ExceptionHandler</code>	Erros como valores	<code>Result</code> + um <code>enum</code> de erro que implementa <code>IntoResponse</code>
Dependências	Contêiner de DI	Structs montadas no <code>main</code>	<code>State<AppState></code> montado no <code>main</code>
Validação	<code>AddValidation</code>	Métodos <code>Validar()</code>	<code>garde</code> ou <code>validator</code> com <code>derive</code> , ou <code>TryFrom</code>
OpenAPI	Gerado do código	Spec-first (oapi-codegen)	Gerado dos tipos (<code>utoipa</code>)
Testes de integração	<code>WebApplicationFactory</code>	<code>httptest</code>	<code>Router</code> + <code>tower::ServiceExt::oneshot</code>

Mapa dos módulos

Módulo	Pergunta que responde	O que você entrega
1. O problema	O que a API garante, e como o ecossistema de Rust se organiza?	Requisitos e a escolha do framework
2. Arquitetura	Que caminho uma requisição percorre?	O mapa de <code>hyper</code> , <code>tower</code> e <code>axum</code>
3. Fundamentos	Como declaro rotas e recebo dados?	Os primeiros handlers com <code>extractors</code> e estado
4. Respostas e erros	Como devolvo sucesso e falha de forma previsível?	<code>ProblemDetails</code> , <code>ErroDaApi</code> e validação

Módulo	Pergunta que responde	O que você entrega
5. Organização	Como o código cresce sem virar um <code>main.rs</code> gigante?	Módulos por feature, extractors e middlewares próprios
6. Dados	Como ler, gravar e receber arquivos sem travar nada?	sqlx, keyset, streaming, upload e idempotência
7. Contrato	Como o cliente sabe o que a API faz?	OpenAPI gerado e versionamento
8. Segurança	Quem pode chamar o quê, e quantas vezes?	JWT, autorização, rate limiting e CORS
9. Performance	Como responder rápido sob carga?	Timeouts, cache, compressão e SSE
10. Testes	Como provo que tudo funciona?	<code>oneshot</code> , banco real e propriedades
11. Operação	Como sei que está saudável em produção?	Desligamento gracioso, telemetria e deploy
Projeto final	Tudo junto funciona?	A API de cobrança completa em Rust

Sugestão de ritmo: um módulo por semana. Os Módulos 2, 3 e 4 concentram o que é específico do axum e merecem mais tempo.

Módulo 1 — O problema: o que a API garante, e como o ecossistema de Rust se organiza

As garantias de uma API de produção são as mesmas em qualquer linguagem: contrato explícito, erros previsíveis, entrada validada, autorização por recurso, idempotência, limites, desempenho, evolução e operabilidade. Em Rust, uma parte surpreendente delas é cumprida pelo **compilador**: a forma da entrada, os desfechos possíveis de cada handler e a segurança do estado compartilhado entre requisições são tipos, e um tipo errado não compila.

1.1 As garantias, e quem cumpre cada uma em Rust

Garantia	Pergunta que ela responde	Onde o curso trata	Em Rust, quem cumpre
Contrato explícito	O cliente sabe o que mandar e o que recebe?	Módulo 7	Os tipos de entrada e saída, com OpenAPI gerado deles
Erros previsíveis	Toda falha tem o mesmo formato?	Módulo 4	Um <code>enum ErroDaApi</code> com uma única implementação de <code>IntoResponse</code>
Entrada validada	Dado inválido é barrado antes da regra?	Módulo 4	Extractors e tipos de domínio que só existem válidos
Autorização	Cada cliente só vê o que é dele?	Módulo 8	Um extractor de identidade + filtro por cliente em toda consulta
Idempotência	Repetir uma chamada duplica efeito?	Módulo 6	Tabela de chaves e <code>ON CONFLICT</code>
Limites	Um cliente pode derrubar os outros?	Módulos 8 e 9	Camadas <code>tower</code> de limite, timeout e tamanho de corpo
Desempenho	A latência aguenta o pico?	Módulos 6 e 9	Consultas certas, streaming, cache
Evolução	Dá para mudar sem quebrar clientes?	Módulo 7	Diff do documento OpenAPI no CI
Operabilidade	Dá para rastrear uma requisição?	Módulo 11	<code>tracing</code> + <code>OpenTelemetry</code> com <code>trace_id</code>

A semântica de HTTP (verbos seguros e idempotentes, status codes certos, `Idempotency-Key` para `POST`) é a do curso de .NET, Módulo 1.2, palavra por palavra.

1.2 Cinco características de Rust que moldam uma API

1. Extractors: a entrada é descrita por tipos. Um handler declara o que precisa nos parâmetros, `Path(id): Path<Uuid>`, `Query(filtro): Query<Filtro>`, `Json(pedido): Json<Pedido>`, e o axum extrai e converte antes de chamar. Se a conversão falha, a requisição é recusada sem que o handler rode. É o model binding do .NET, só que a correspondência entre tipo e origem é verificada em compilação.

2. Erros são `Result`, e respostas de erro são um tipo seu. Um handler devolve `Result<Json<T>, ErroDaApi>`, e o operador `?` propaga qualquer erro que saiba se converter em `ErroDaApi`. Quem decide o status e o corpo de cada erro é uma única implementação de `IntoResponse` (Módulo 4), o equivalente ao `ExceptionHandler` sem exceções.

3. O estado compartilhado é tipado e precisa ser seguro entre threads. O pool do banco, o validador de JWT e o cache vão num `AppState` passado ao `Router`; os handlers pedem `State(estados): State<AppState>`. O compilador exige que o estado seja `Clone + Send + Sync`, então um campo que não pode ser compartilhado entre requisições concorrentes simplesmente não entra.

4. Handlers são futures que precisam ser `Send`. O tokio pode continuar um handler em outra thread depois de cada `.await`. Segurar algo que não é `Send` (um `std::sync::MutexGuard`, um `Rc`) através de um `.await` produz um dos erros de compilação mais longos do axum: "the trait `Handler<_, _>` is not implemented". A macro `#[axum::debug_handler]` traduz esse erro para a linha que o causou; use-a sempre que um handler não compilar.

5. O corpo da requisição é consumido uma vez. Extractors que leem o corpo (`Json`, `Bytes`, `Body`, `Multipart`) precisam ser o **último** parâmetro do handler, e só pode haver um. O compilador impõe a regra: os outros extractors implementam `FromRequestParts` (só olham cabeçalhos, rota e query), e apenas o último pode implementar `FromRequest`.

1.3 O ecossistema web de Rust

Framework	O que é	Base	Quando escolher
axum	Roteador e extractors sobre <code>hyper</code> , mantido pelo time do tokio	<code>tower</code> : todo middleware é um <code>Service</code>	Padrão para APIs novas; este curso
actix-web	Framework maduro e muito rápido, com runtime próprio sobre o tokio	Middlewares próprios	Bases existentes em actix; quem quer o ecossistema dele
Poem (com <code>poem-openapi</code>)	Framework com OpenAPI de primeira classe	Middlewares próprios	Quando o contrato gerado é o centro do projeto
Rocket	Framework com foco em ergonomia e macros	Própria	Projetos pequenos, protótipos

Framework	O que é	Base	Quando escolher
Loco	Camada "estilo Rails" sobre o axum	axum	Aplicações web completas com convenções prontas

O critério decisivo é a terceira coluna, o mesmo argumento do curso de Go sobre compatibilidade com `http.Handler`: no axum, todo middleware é um `tower::Service`, e o ecossistema `tower-http` já oferece trace, compressão, CORS, timeout, limite de corpo e propagação de IDs prontos e testáveis. O mesmo middleware serve para clientes HTTP (`request` e `tonic` também são `tower`).

1.4 Os requisitos da API de cobrança

Os mesmos endpoints, agora na sintaxe de rotas do axum 0.8 (curingas entre chaves, como no Go):

Rota	O que faz	Detalhe que torna difícil
<code>POST /v1/arquivos</code>	Recebe um arquivo de retorno	Upload grande em streaming, idempotência, 202
<code>GET /v1/arquivos/{id}</code>	Status do processamento	Autorização: só o dono vê
<code>GET /v1/arquivos/{id}/progresso</code>	Progresso em tempo real	Server-Sent Events
<code>GET /v1/arquivos</code>	Lista arquivos do cliente	Paginação keyset, filtros
<code>GET /v1/titulos/{nosso_numero}</code>	Consulta um título	Cache, 404 correto
<code>GET /v1/titulos/{nosso_numero}/ocorrencias</code>	Histórico do título	Lista potencialmente longa
<code>GET /v1/ocorrencias/export</code>	Exporta ocorrências do dia	Resposta em streaming de milhões de linhas

Requisitos não funcionais iguais aos das outras duas APIs: p95 abaixo de 200 ms para consultas, uploads de até 500 MB, 100 requisições por minuto por cliente e JWT emitido por um provedor de identidade externo.

Exercício do Módulo 1

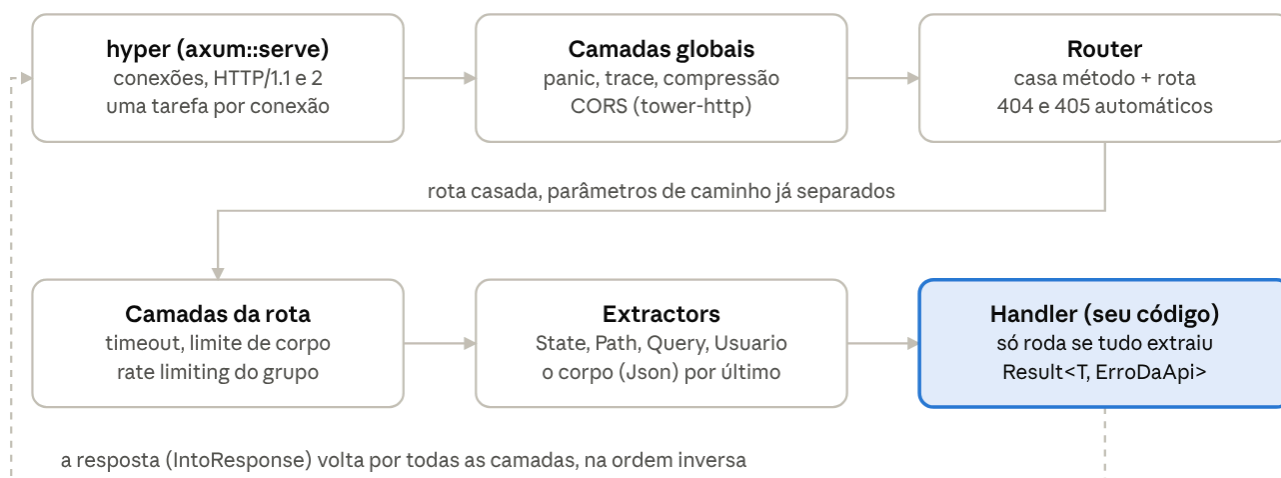
Crie o projeto com `cargo new cobranca-api`, adicione `axum`, `tokio` (feature `full`) e `serde`, e escreva um handler que recebe `Path<Uuid>` e devolve `"ok"`. Depois quebre-o de três jeitos e leia os erros: um `Json<T>` antes do `Path` na lista de parâmetros; dois extractors de corpo; e um `std::sync::Mutex` travado através de um `.await`. Repita cada um com `#[axum::debug_handler]` e compare as mensagens.

Módulo 2 — Arquitetura: o caminho de uma requisição no axum

O axum é uma camada fina sobre duas peças: o **hyper**, que fala HTTP, e o **tower**, que define a abstração `Service` (uma função assíncrona de requisição para resposta) e a forma de empilhar middlewares como `Layer`s. O próprio `Router` é um `Service`. Entre o router e o seu código ficam os **extractors**, que convertem a requisição nos parâmetros tipados do handler.

No axum, a requisição atravessa camadas tower até os extractors e o handler

Caminho de uma requisição · cada camada e o próprio Router são um `tower::Service`



caminho de uma requisição no axum · 6 etapas

A forma é a mesma das outras duas APIs (ida por camadas, volta na ordem inversa). O que é do axum é a quinta caixa: os extractors são executados em ordem, e o handler só é chamado se todos tiverem sucesso.

2.1 O servidor

```
#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let estado = AppState::montar(Config::do_ambiente()).await?;
    let app = montar_router(estado);

    let listener = tokio::net::TcpListener::bind("0.0.0.0:8080").await?;
    axum::serve(listener, app)
        .with_graceful_shutdown(sinal_de_desligamento()) // Módulo 11
        .await?;
    Ok(())
}
```

`axum::serve` aceita conexões e entrega cada uma ao `hyper`, numa tarefa do `tokio`. Sobre timeouts, há uma diferença importante em relação ao `http.Server` do Go: o

`axum::serve` expõe poucas opções de conexão. O prazo para o processamento de cada requisição vem de uma camada (`TimeoutLayer`, seção 2.2); a proteção contra clientes que enviam cabeçalhos lentamente fica a cargo do `hyper` (que aplica um prazo de leitura de cabeçalhos nas versões recentes) e, em produção, do ingress na frente da API. Para controle fino da conexão, é possível montar o servidor direto com `hyper-util`.

2.2 Service, Layer e a ordem das camadas

Um `Layer` embrulha um `Service` e devolve outro, exatamente como o

`func(http.Handler) http.Handler` do Go. O `ServiceBuilder` empilha camadas numa ordem legível, com a **primeira listada como a mais externa**:

```

use tower::ServiceBuilder;
use tower_http::{catch_panic::CatchPanicLayer, compression::CompressionLayer,
trace::TraceLayer};

fn montar_router(estados: AppState) -> Router {
    Router::new()
        .nest("/v1", rotas_v1(estados.clone())) // as camadas de prazo e corpo
        ficam dentro (Módulo 5)
        .route("/health/live", get(saude::vivo))
        .route("/health/ready", get(saude::pronto))
        .layer(
            ServiceBuilder::new()
                .layer(CatchPanicLayer::custom(errores::responder_panico)) //
1. o mais externo: pega panics
                .layer(TraceLayer::new_for_http()) // 2.
um span por requisição
                .layer(CompressionLayer::new()), // 3.
comprime a resposta
            )
            .with_state(estados)
        }
}

```

Duas regras do `Router` que pegam todo mundo uma vez:

- **`.layer(...)` só se aplica às rotas registradas antes dele.** Uma rota adicionada depois do `.layer` fica sem aquela camada. Registre todas as rotas primeiro e as camadas por último.
- **`.layer` x `.route_layer`.** `.layer` envolve o router inteiro, inclusive o 404 de uma rota que não existe; `route_layer` só roda quando alguma rota casou. Autenticação vai em `route_layer`: uma URL inexistente deve dar 404, não 401.

E uma regra de desenho que repete a lição do curso de Go sobre prazos: **limites que algumas rotas precisam afrouxar não podem ficar na camada global.** Uma `TimeoutLayer` de 10 segundos por fora de tudo cortaria o upload de 500 MB, a exportação e o stream de progresso, e uma camada interna não consegue alongar um prazo que a externa impõe. Por isso o prazo e o limite de corpo moram nos sub-routers de cada grupo de rotas (Módulo 5), e não na pilha acima.

2.3 Middleware em uma função

Para middlewares próprios, `middleware::from_fn` aceita uma função assíncrona comum, sem implementar `Service` à mão:

```

use axum::{extract::{MatchedPath, Request}, middleware::Next,
response::Response};

async fn registrar(req: Request, next: Next) -> Response {
    let inicio = Instant::now();
    let metodo = req.method().clone();
    let rota = req.extensions().get::().map(|p|
p.as_str().to_owned()); // "/v1/arquivos/{id}"

    let resposta = next.run(req).await; // todo o resto da cadeia

    tracing::info!(
        %metodo,
        rota = rota.as_deref().unwrap_or("desconhecida"),
        status = resposta.status().as_u16(),
        duracao_ms = inicio.elapsed().as_millis() as u64,
        "requisição"
    );
    resposta
}

```

`MatchedPath` é o padrão de rota que casou, o equivalente ao `r.Pattern` do Go e ao template de rota do .NET: é ele, e não a URL com IDs, que vai para logs e métricas. Na prática, o `TraceLayer` do `tower-http` já faz este trabalho, configurável; o exemplo está aqui para mostrar o formato.

2.4 Rotas

```

Router::new()
    .route("/arquivos", get(listar).post(receber))
    .route("/arquivos/{id}", get(obter))
    .route("/arquivos/{id}/progresso", get(progresso))
    .route("/arquivos-brutos/{*chave}", get(baixar)) // curinga que pega o
resto do caminho

```

- **Curingas entre chaves** (`{id}`) desde o axum 0.8; a sintaxe antiga `:id` foi removida.
- **Método errado devolve 405** e rota inexistente devolve 404, ambos automáticos e personalizáveis com `fallback` e `method_not_allowed_fallback`, que é onde o Módulo 4 coloca o `ProblemDetails`.
- **Rotas que se sobrepõem causam panic** na montagem do router, ou seja, na subida, como no `ServeMux` do Go.
- **`nest("/v1", outro)`** monta um router sob um prefixo, e **`merge`** junta dois routers no mesmo nível. São as peças de organização do Módulo 5.

2.5 Extractors

Cada parâmetro do handler é um extractor, e o axum os executa da esquerda para a direita:

```
async fn obter(
    State(estado): State<AppState>, // o estado compartilhado
    usuario: Usuario,               // extractor próprio: a identidade do
    token (Módulo 8)
    Path(id): Path<Uuid>,           // o curinga {id}, já convertido
) -> Result<Json<ArquivoResposta>, ErroDaApi> {
    // ...
}
```

Há dois tipos de extractor: os que implementam `FromRequestParts` (olham método, URI, cabeçalhos e extensões, e podem ser quantos forem) e os que implementam `FromRequest` (consomem o corpo, só um, sempre por último). Se um extractor falha, ele devolve uma *rejection* (por exemplo, 400 para um UUID inválido no caminho) e o handler nem roda. Para tratar a falha você mesmo, peça `Result<Path<Uuid>, PathRejection>` ou, quando o dado é opcional, `Option<...>`. O Módulo 4 troca as rejections padrão (texto puro) por `ProblemDetails`.

2.6 Estado compartilhado

O `AppState` é uma struct `Clone` com o que as requisições compartilham, passada uma vez com `.with_state(estado)`. Como ele é clonado a cada requisição, os campos são baratos de clonar: o pool do `sqlx` já é um ponteiro com contagem de referências, e o resto vai em `Arc`:

```
#[derive(Clone)]
pub struct AppState {
    pub banco: sqlx::PgPool,           // já é barato de clonar
    pub jwt: Arc<auth::Validador>,
    pub storage: Arc<dyn armazenamento::Storage>,
    pub limites: Arc<limites::PorCliente>,
}
```

Handlers que precisam só de uma parte podem pedir só aquela parte, implementando `FromRef<AppState>` para o tipo (por exemplo, `State(pool): State<PgPool>`). Existe também o extractor `Extension`, verificado só em tempo de execução; prefira `State`, que o compilador confere.

2.7 Dados de cada requisição

Não há escopo de DI por requisição. O que pertence a uma requisição específica (a identidade autenticada, um ID de correlação) vai nas **extensões** da requisição: um middleware as insere (`req.extensions_mut().insert(identidade)`) e um extractor as lê. É o papel do `context.Context` do Go para valores, com tipos em vez de chaves. Cancelamento não precisa de token: quando o cliente desconecta, o hyper descarta a future do handler, e o trabalho para no próximo `.await`. Com as ressalvas do curso de arquivos em Rust (Módulo 7.6): descartar a future não cancela uma consulta já enviada ao PostgreSQL, e código que precisa terminar (como limpar um objeto enviado ao storage) não pode depender de chegar ao fim do handler.

Exercício do Módulo 2

Monte o router de 2.2 com o middleware `registrar` de 2.3 e duas rotas, `GET /v1/arquivos/{id}` e `GET /v1/arquivos/{id}/progresso`. Confirme no log que aparece a rota com `{id}`, não o UUID. Faça um `POST` numa rota só de `GET` (405) e uma URL inexistente (404). Registre uma rota depois do `.layer(...)` e confirme que ela não é registrada pelo middleware. Por fim, registre duas rotas conflitantes e leia o `panic` na subida.

Módulo 3 — Fundamentos: rotas, extractors e serde

No axum, a entrada de uma requisição é descrita por tipos: um handler pede `Path<NossoNumero>`, e a conversão e a validação do curinga acontecem antes de ele rodar. O trabalho deste módulo é fazer os tipos certos: tipos de domínio que só existem válidos, structs de entrada que recusam o que não conhecem e structs de saída cujos nomes de campo são o contrato. Quase tudo isso é `serde`.

3.1 Configuração que falha cedo

```
pub struct Config {
    pub endereco: SocketAddr,
    pub banco_url: String,
    pub jwt_emissor: String,
    pub jwt_audiencia: String,
}

impl Config {
    pub fn do_ambiente() -> anyhow::Result<Self> {
        let obrigatoria = |nome: &str| {
            std::env::var(nome).with_context(|| format!("variável obrigatória
ausente: {nome}"))
```

```

};
Ok(Self {
    endereco: std::env::var("ENDERECO")
        .unwrap_or_else(|_| "0.0.0.0:8080".into())
        .parse()
        .context("ENDERECO inválido")?,
    banco_url: obrigatoria("BANCO_URL")?,
    jwt_emissor: obrigatoria("JWT_EMISSOR")?,
    jwt_audiencia: obrigatoria("JWT_AUDIENCIA")?,
})
}
}

```

`with_context` e `context` (do `anyhow`) acrescentam uma frase ao erro, e o `main` que devolve `anyhow::Result` imprime a cadeia inteira e sai com código de falha: configuração errada impede a subida, o mesmo papel do `ValidateOnStart` do .NET e do `carregarConfig` do Go.

3.2 Os extractors que você vai usar

Extractor	Origem	Observação
<code>Path<T></code>	Curingas da rota	<code>T</code> pode ser um tipo, uma tupla ou uma struct com os nomes dos curingas
<code>Query<T></code>	Query string	<code>T</code> é uma struct <code>Deserialize</code> ; campos ausentes precisam de <code>Option</code> ou <code>default</code>
<code>HeaderMap</code>	Todos os cabeçalhos	Para um cabeçalho tipado, <code>TypedHeader</code> do <code>axum-extra</code>
<code>State<T></code>	O estado da aplicação	Verificado em compilação
<code>MatchedPath</code>	O padrão de rota que casou	Para logs e métricas
<code>Json<T></code>	Corpo JSON	Consome o corpo: sempre por último
<code>Bytes</code> / <code>String</code>	Corpo inteiro na memória	Sujeito ao <code>DefaultBodyLimit</code> (2 MB por padrão)
<code>Body</code>	Corpo em streaming	Para uploads grandes (Módulo 6)
Extractor próprio	O que você quiser	<code>Usuario</code> , <code>ChaveDeIdempotencia</code> (Módulos 5 e 8)

3.3 Tipos de domínio que só existem válidos

O `Path` usa o `serde` para converter o curinga. Com `#[serde(try_from = "String")]`, a conversão passa pela sua função de validação, e um `NossoNumero` inválido nunca chega a existir:

```
use serde::{Deserialize, Serialize};
use std::str::FromStr;

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]
#[serde(try_from = "String", into = "String")]
pub struct NossoNumero(i64);

#[derive(Debug, thiserror::Error)]
#[error("nosso número deve ter de 1 a 18 dígitos e não ser zero")]
pub struct NossoNumeroInvalido;

impl FromStr for NossoNumero {
    type Err = NossoNumeroInvalido;
    fn from_str(s: &str) -> Result<Self, Self::Err> {
        // 18 dígitos sempre cabem em i64: sem risco de estouro
        if s.is_empty() || s.len() > 18 || !s.bytes().all(|b|
b.is_ascii_digit()) {
            return Err(NossoNumeroInvalido);
        }
        match s.parse::<i64>() {
            Ok(0) | Err(_) => Err(NossoNumeroInvalido),
            Ok(v) => Ok(Self(v)),
        }
    }
}

impl TryFrom<String> for NossoNumero {
    type Error = NossoNumeroInvalido;
    fn try_from(s: String) -> Result<Self, Self::Error> { s.parse() }
}

impl From<NossoNumero> for String {
    fn from(n: NossoNumero) -> Self { n.0.to_string() }
}

impl NossoNumero {
    pub fn valor(self) -> i64 { self.0 }
}
```

O campo interno é privado: fora do módulo, a única forma de obter um `NossoNumero` é pelo parse. Com `into = "String"`, ele também vira texto na saída JSON, e o mesmo tipo vale para rota, query e corpo. Uma rota declarada como `/v1/titulos/{nosso_numero}` com um handler que pede `Path(numero): Path<NossoNumero>` devolve 400 para `abc` e `0` sem uma linha de código no handler; o Módulo 4 transforma essa rejeição em `ProblemDetails`.

O status do arquivo é um `enum` cujos nomes no JSON são os valores da coluna `status`:

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize,
sqlx::Type)]
#[serde(rename_all = "SCREAMING_SNAKE_CASE")]
#[sqlx(type_name = "text", rename_all = "SCREAMING_SNAKE_CASE")]
pub enum StatusArquivo { Recebido, Processando, Concluido, Rejeitado, Falhou
}

impl StatusArquivo {
    /// O texto gravado na coluna status (usado como parâmetro de consulta no
    Módulo 6)
    pub const fn como_texto(self) -> &'static str {
        match self {
            Self::Recebido => "RECEBIDO",
            Self::Processando => "PROCESSANDO",
            Self::Concluido => "CONCLUIDO",
            Self::Rejeitado => "REJEITADO",
            Self::Falhou => "FALHOU",
        }
    }
}
```

O mesmo `enum` serve para o JSON (pelo `serde`) e para o banco (pelo `sqlx::Type`, Módulo 6). Um valor desconhecido num filtro vira 400; um valor desconhecido vindo do banco vira erro de decodificação, que é o comportamento certo se alguém inventar um status novo sem atualizar a API.

3.4 Filtros de listagem

```
#[derive(Debug, Deserialize)]
#[serde(deny_unknown_fields)]
pub struct FiltroDeArquivos {
    pub status: Option<StatusArquivo>,
    pub de: Option<NaiveDate>,      // "AAAA-MM-DD" (feature serde do chrono)
    pub ate: Option<NaiveDate>,
    pub cursor: Option<String>,
    #[serde(default = "limite_padrao")]
    pub limite: u32,
}

fn limite_padrao() -> u32 { 50 }
```

A diferença entre ausente e inválido sai de graça: sem `limite` na query, vale 50; com `limite=x` ou `limite=` (vazio), o `Query` falha com 400. `deny_unknown_fields` faz um `?limit=10` (em inglês, por engano) virar erro em vez de ser ignorado. Regras entre campos (de antes de `ate`, intervalo máximo) ficam na validação do Módulo 4.

3.5 JSON de entrada

`Json<T>` lê o corpo (até o `DefaultBodyLimit`, 2 MB por padrão), confere o `Content-Type: application/json` e desserializa. Os padrões do `serde_json` já são rigorosos: um campo duplicado num objeto é erro, e desde o axum 0.8.5 caracteres depois do fim do documento JSON também são recusados. Acrescente `#[serde(deny_unknown_fields)]` nas structs de entrada para recusar campos com nome errado, com a mesma ressalva dos outros cursos: APIs públicas que evoluem às vezes preferem tolerar campos desconhecidos; decida e documente.

Uma diferença de Rust em relação ao Go: um campo obrigatório é simplesmente um campo que não é `Option`. Se ele faltar no JSON, a desserialização falha; não existe "valor zero" que passe em silêncio.

3.6 JSON de saída

```
#[derive(Debug, Serialize)]
#[serde(rename_all = "camelCase")]
pub struct ArquivoResposta {
    pub id: Uuid,
    pub nome_original: String,
    pub banco: String,
    pub status: StatusArquivo,
    pub recebido_em: DateTime<Utc>,
    pub linhas_processadas: i32,
    pub linhas_rejeitadas: i32,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub motivo: Option<String>,
}
```

`rename_all = "camelCase"` transforma `nome_original` em `nomeOriginal`, mantendo os nomes Rust idiomáticos e o contrato igual ao das APIs em .NET e Go. `Uuid` e `DateTime<Utc>` saem como texto (UUID em minúsculas com hífens, data em RFC 3339). `skip_serializing_if` omite `motivo` quando não há motivo, como o `omitzero` do Go. A struct de resposta é o contrato; a do banco pode mudar sem quebrá-lo.

3.7 Um handler completo

Ainda sem o `ErroDaApi` do Módulo 4, usando `StatusCode` como erro (que já implementa `IntoResponse`):

```
async fn obter_titulo(
    State(estados): State<AppState>,
    Path(numero): Path<NossoNumero>, // já validado: 400 automático para
    entradas inválidas
) -> Result<Json<TituloResposta>, StatusCode> {
    match estados.titulos.obter(numero).await {
        Ok(Some(titulo)) => Ok(Json(titulo)),
        Ok(None) => Err(StatusCode::NOT_FOUND),
        Err(e) => {
            tracing::error!(erro = %e, "falha ao consultar título");
            Err(StatusCode::INTERNAL_SERVER_ERROR)
        }
    }
}

// registro
Router::new().route("/v1/titulos/{nosso_numero}", get(obter_titulo))
```

Repare no que não está no handler: nenhuma leitura de curinga, nenhum parse, nenhum `return` esquecido depois de escrever um erro (o tipo de retorno obriga a devolver exatamente uma resposta). O `Json(titulo)` define o `Content-Type` e serializa; se a serialização falhar, o próprio axum responde 500.

Exercício do Módulo 3

Implemente `GET /v1/titulos/{nosso_numero}` com um repositório em memória e teste quatro chamadas: número válido existente (200), válido inexistente (404), `abc` (400) e `0` (400). Leia o corpo das respostas 400 e note que são texto puro, não JSON: o Módulo 4 resolve. Depois implemente `GET /v1/arquivos` com `Query<FiltroDeArquivos>` e confira que `?limite=x` e `?limite=` geram 400, `?limit=10` também, e a ausência de `limite` usa 50.

Módulo 4 — Respostas e erros previsíveis

Em Rust, o "tratador central de erros" é um tipo: um `enum ErroDaApi` com uma variante por desfecho de erro e uma implementação de `IntoResponse` que decide status e corpo de cada uma. Os handlers devolvem `Result<_, ErroDaApi>`, o operador `?` converte qualquer erro conhecido, e o compilador garante que nenhuma variante fica sem resposta. As rejeições dos extractors (o 400 em texto puro do Módulo 3) passam pelo mesmo caminho.

4.1 IntoResponse: qualquer coisa vira resposta

Um handler pode devolver qualquer tipo que implemente `IntoResponse`: `Json<T>`, `StatusCode`, `String`, e também tuplas que combinam status, cabeçalhos e corpo:

```
async fn receber(/* ... */) -> Result<impl IntoResponse, ErroDaApi> {
    let arquivo = /* ... */;
    let local = format!("/v1/arquivos/{})", arquivo.id);
    Ok((StatusCode::ACCEPTED, [(header::LOCATION, local)], Json(arquivo))) //
    202 + Location + corpo
}
```

O tipo de retorno é o contrato do lado de sucesso. `Result<Json<ArquivoResposta>, ErroDaApi>` diz exatamente o que o `Results<Ok<ArquivoResposta>, NotFound>` do .NET dizia, com a diferença de que o conjunto de erros possíveis está no `enum` (o Módulo 7 usa isso para gerar o OpenAPI).

4.2 ProblemDetails

O mesmo formato da RFC 9457 das outras duas APIs:

```
#[derive(Debug, Serialize)]
#[serde(rename_all = "camelCase")]
pub struct ProblemDetails {
    #[serde(rename = "type")]
    pub tipo: String,
    pub title: String,
    pub status: u16,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub detail: Option<String>,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub instance: Option<String>,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub trace_id: Option<String>,
    #[serde(skip_serializing_if = "BTreeMap::is_empty")]
    pub errors: BTreeMap<String, Vec<String>>,
    #[serde(flatten)]
    pub extensoes: serde_json::Map<String, serde_json::Value>, // campos
    extras no mesmo nível
}
```

`#[serde(flatten)]` coloca as extensões (como `arquivoExistenteId`) ao lado de `title` e `status`, como o padrão pede. O `BTreeMap` mantém as chaves de `errors` em ordem alfabética, o que deixa as respostas estáveis e os testes de snapshot (Módulo 10) previsíveis.

4.3 O ErroDaApi

```
#[derive(Debug, thiserror::Error)]
pub enum ErroDaApi {
    #[error("recurso não encontrado")]
    NaoEncontrado,
    #[error("requisição inválida")]
    Validacao(BTreeMap<String, Vec<String>>),
    #[error("requisição malformada: {0}")]
    Requisicao(String),
    #[error("corpo da requisição grande demais")]
    CorpoGrandeDemais,
    #[error("este arquivo já foi recebido")]
    ArquivoDuplicado { existente: Uuid },
    #[error("layout de arquivo não suportado: {0}")]
    LayoutNaoSuportado(String),
    #[error("não autenticado")]
    NaoAutenticado,
    #[error("escopo insuficiente")]
    Proibido,
}
```

```

    #[error("limite de requisições excedido")]
    LimiteExcedido { tentar_em_segundos: u64 },
    #[error(transparent)]
    Interno(#[from] anyhow::Error),
}

const BASE: &str = "https://api.cobranca.exemplo/erros/";

impl IntoResponse for ErroDaApi {
    fn into_response(self) -> Response {
        let mut p = ProblemDetails::novo(self.status(), &self.to_string());
        match &self {
            Self::Validacao(erros) => {
                p.tipo = format!("{BASE}validacao");
                p.errors = erros.clone();
            }
            Self::ArquivoDuplicado { existente } => {
                p.tipo = format!("{BASE}arquivo-duplicado");
                p.extensoes.insert("arquivoExistenteId".into(),
existente.to_string().into());
            }
            Self::LayoutNaoSuportado(_) => p.tipo = format!("{BASE}layout-
nao-suportado"),
            Self::Requisicao(detalle) => p.detail = Some(detalle.clone()),
            Self::Interno(e) => {
                tracing::error!(erro = ?e, "erro não tratado"); // a cadeia
completa só no log
                p.title = "Erro interno.".into(); // nunca na
resposta
            }
            Self::NaoEncontrado | Self::CorpoGrandeDemais |
Self::NaoAutenticado
            | Self::Proibido | Self::LimiteExcedido { .. } => {}
        }

        let mut resposta = p.into_response(); // application/problem+json,
com traceId e instance
        match self {
            Self::NaoAutenticado => {
                resposta.headers_mut().insert(header::WWW_AUTHENTICATE,
HeaderValue::from_static("Bearer"));
            }
            Self::LimiteExcedido { tentar_em_segundos } => {
                resposta.headers_mut().insert(header::RETRY_AFTER,
HeaderValue::from(tentar_em_segundos));
            }
        }
    }
}

```

```

        _ => {}
    }
    resposta
}

impl ErroDaApi {
    fn status(&self) -> StatusCode {
        match self {
            Self::NaoEncontrado => StatusCode::NOT_FOUND,
            Self::Validacao(_) | Self::Requisicao(_) =>
                StatusCode::BAD_REQUEST,
            Self::CorpoGrandeDemais => StatusCode::PAYLOAD_TOO_LARGE,
            Self::ArquivoDuplicado { .. } => StatusCode::CONFLICT,
            Self::LayoutNaoSuportado(_) => StatusCode::UNPROCESSABLE_ENTITY,
            Self::NaoAutenticado => StatusCode::UNAUTHORIZED,
            Self::Proibido => StatusCode::FORBIDDEN,
            Self::LimiteExcedido { .. } => StatusCode::TOO_MANY_REQUESTS,
            Self::Interno(_) => StatusCode::INTERNAL_SERVER_ERROR,
        }
    }
}

```

Por que esse desenho:

- **Exaustividade.** Os dois `match` (o de `status` e o de `into_response`) precisam cobrir todas as variantes. Acrescentar `ErroDaApi::TituloLiquidado` sem decidir seu status não compila. Em .NET e Go, esquecer um caso no mapeamento central dá um 500 em produção.
- **Desfechos de negócio como variantes.** `ArquivoDuplicado { existente }` carrega o dado que o `ProblemDetails` precisa, tipado. Não há o `map[string]any` de extensões do Go no código de negócio, só na borda.
- **`Interno(#[from] anyhow::Error)`.** Qualquer erro inesperado, de qualquer biblioteca, vira 500 com `?` (desde que possa virar `anyhow::Error`), é registrado com a cadeia completa de causas e nunca vaza na resposta.
- **`traceId` e `instance`.** O `ProblemDetails::into_response` preenche o `traceId` a partir do span atual do `tracing` (o `TraceLayer` mantém o span da requisição ativo enquanto o handler e a conversão da resposta rodam) e o `instance` a partir de um `tokio::task::local!` com método e caminho, preenchido por um middleware de duas linhas que envolve `next.run(req)` no escopo desse valor.

4.4 Rejeições dos extractors no mesmo formato

Os extractors padrão rejeitam com texto puro. Para que um UUID inválido no caminho ou um JSON malformatado também saiam como `ProblemDetails`, crie versões dos extractors que usam `ErroDaApi` como rejeição, com a macro `FromRequest` / `FromRequestParts` do `axum`:

```
#[derive(FromRequest)]
#[from_request(via(axum::Json), rejection(ErroDaApi))]
pub struct Json<T>(pub T);

#[derive(FromRequestParts)]
#[from_request(via(axum::extract::Path), rejection(ErroDaApi))]
pub struct Path<T>(pub T);

#[derive(FromRequestParts)]
#[from_request(via(axum::extract::Query), rejection(ErroDaApi))]
pub struct Query<T>(pub T);

impl From<JsonRejection> for ErroDaApi {
    fn from(r: JsonRejection) -> Self {
        if r.status() == StatusCode::PAYLOAD_TOO_LARGE {
            Self::CorpoGrandeDemais
        } else {
            Self::Requisicao(r.body_text())
        }
    }
}

// ... e o mesmo para PathRejection e QueryRejection
```

Nos handlers, importe `crate::extract::{Json, Path, Query}` em vez dos do `axum`, e o resto do código não muda. Para o 404 de rota inexistente e o 405 de método errado, registre no router `fallback` e `method_not_allowed_fallback` com handlers que devolvem `ErroDaApi::NaoEncontrado` e um `ProblemDetails` de 405.

4.5 Validação: um extractor Validado

Rust oferece duas abordagens, e as duas têm lugar:

- **"Parse, don't validate"**: tipos como `NossoNumero` (Módulo 3.3), que só existem válidos. Ideal para valores com regra própria que aparecem em muitos lugares.
- **Validação declarativa** com o crate `garde` (ou `validator`): atributos nos campos, verificados por um `derive`, mais funções próprias para regras entre campos.

```
#[derive(Debug, Deserialize, garde::Validate)]
#[serde(deny_unknown_fields)]
pub struct RegistrarCallbackRequest {
    #[garde(url, length(max = 500), custom(https))]
    pub url: String,
    #[garde(length(min = 1))]
    pub eventos: Vec<String>,
}

fn https(url: &str, _: &()) -> garde::Result {
    if url.starts_with("https://") { Ok(()) } else {
        Err(garde::Error::new("deve usar https")) }
}
```

É um extractor junta leitura e validação, para que o handler só receba o que já é válido:

```
pub struct Validado<T>(pub T);

impl<S, T> FromRequest<S> for Validado<T>
where
    S: Send + Sync,
    T: DeserializeOwned + garde::Validate<Context = ()>,
{
    type Rejection = ErroDaApi;

    async fn from_request(req: Request, estado: &S) -> Result<Self,
Self::Rejection> {
        let Json(valor) = Json::<T>::from_request(req, estado).await?; // o
        Json acima, já com ErroDaApi
        valor.validate().map_err(|relatorio| {
            let mut erros = BTreeMap::<String, Vec<String>>::new();
            for (caminho, erro) in relatorio.iter() {

erros.entry(caminho.to_string()).or_default().push(erro.to_string());
            }
            ErroDaApi::Validacao(erros)
        })?;
        Ok(Validado(valor))
    }
}

// uso
async fn registrar_callback(usuario: Usuario, Validado(pedido):
Validado<RegistrarCallbackRequest>) { /* ... */ }
```

No axum 0.8, `FromRequest` usa `async fn` direto no trait, sem a macro `#[async_trait]` das versões anteriores. Para `Query`, um `ValidadoQuery<T>` igual, implementando `FromRequestParts`, valida os filtros de listagem (como `de` antes de `ate`). A separação entre validação de forma (400, sem banco) e regra de negócio (409 ou 422, no serviço) é a mesma dos outros cursos.

4.6 Panic

Um `panic` num handler é um bug. O hyper isola a tarefa e o processo continua, mas o cliente receberia a conexão fechada. O `CatchPanicLayer` do Módulo 2 transforma isso num 500 em `ProblemDetails`:

```
pub fn responder_panico(detalle: Box<dyn Any + Send + 'static>) -> Response {
    let mensagem = detalle
        .downcast_ref::<&str>()
        .map(|s| s.to_string())
        .or_else(|| detalle.downcast_ref::()
            .cloned())
        .unwrap_or_else(|| "panic sem mensagem".into());
    tracing::error!(%mensagem, "panic no handler");
    ErroDaApi::Interno(anyhow::anyhow!("panic")).into_response()
}
```

Como no curso de arquivos em Rust, mantenha `panic = "unwind"` (o padrão) no perfil de release: com `abort`, um `panic` derruba o processo inteiro e esta camada não tem o que capturar.

Exercício do Módulo 4

Implemente `ProblemDetails`, `ErroDaApi`, os extractors `Json`, `Path` e `Query` com rejeição própria, o `Validado<T>` e os `fallback`s do router. Faça chamadas que produzam 400 (UUID inválido e corpo com campo desconhecido), 404 (rota e recurso inexistentes), 405, 409, 413 e 500 (um `panic` proposital) e confirme que todas têm `Content-Type: application/problem+json`, o mesmo formato e um `traceId`. Depois acrescente uma variante a `ErroDaApi` e veja o compilador apontar os dois `match` incompletos. Por fim, compare as respostas com as das APIs em .NET e Go.

Módulo 5 — Organização do código

Uma API em axum se mantém legível quando cada feature é um módulo que expõe uma função `rotas()` devolvendo um `Router<AppState>`; quando grupos de rotas recebem as camadas de que precisam (prazo, limite de corpo, autenticação) em vez de uma pilha global; e quando regras de acesso viram **extractors**, de modo que o próprio tipo de um parâmetro prove que a regra foi verificada.

5.1 Layout do crate

```

cobranca-api/
├─ Cargo.toml
├─ migrations/
retorno_detalhe, outbox...)
├─ .sqlx/
(Módulo 6)
├─ src/
  ├─ main.rs
  ├─ app.rs
  ├─ erros.rs
  ├─ extract.rs
  ├─ auth.rs
8)
  ├─ dominio.rs
  ├─ arquivos/
    │ ├─ mod.rs
    │ ├─ handlers.rs
    │ ├─ contratos.rs
    │ ├─ acesso.rs
    │ ├─ consulta.rs
    │ └─ recebimento.rs
  ├─ titulos/
  └─ ocorrencias/

```

as mesmas tabelas dos workers (arquivo,

cache das consultas para compilar sem banco

main: configuração, telemetria, serve AppState e montar_router: a composição ProblemDetails, ErroDaApi

Json, Path, Query, Validado (Módulo 4)

Usuario, validador de JWT, escopos (Módulo

NossoNumero, StatusArquivo

rotas(): o índice das rotas da feature

structs de entrada e saída

o extractor ArquivoDoCliente

leituras

regra do upload

Módulos por feature, não por camada, como nos outros cursos. Um módulo só expõe o que for `pub`, e a feature `arquivos` expõe basicamente `rotas()`. Quando o crate crescer a ponto de a compilação incomodar, as features podem virar crates de um workspace: o compilador do Rust paraleliza por crate, e mudar um handler deixa de recompilar a API inteira.

5.2 Uma feature: rotas() e handlers

```

// src/arquivos/mod.rs
pub fn rotas_json() -> Router<AppState> {
    Router::new()
        .route("/arquivos", get(handlers::listar))
        .route("/arquivos/{id}", get(handlers::obter))
}

/// Rotas que precisam de limites próprios: upload grande e stream de
progresso
pub fn rotas_longas() -> Router<AppState> {
    Router::new()

```

```

        .route("/arquivos", post(handlers::receber))
        .route("/arquivos/{id}/progresso", get(handlers::progresso))
    }

// src/arquivos/handlers.rs
pub async fn obter(
    State(estados): State<AppState>,
    arquivo: ArquivoDoCliente, // só existe se o arquivo for do cliente
    autenticado (5.4)
) -> Result<Json<ArquivoResposta>, ErroDaApi> {
    let resposta = consulta::obter(&estados.banco,
    &arquivo).await?.ok_or(ErroDaApi::NaoEncontrado)?;
    Ok(Json(resposta))
}

```

O `Router<AppState>` ainda não tem o estado: ele declara de que estado precisa, e o `with_state` no topo fornece. Se uma feature pedir um estado que a composição não fornece, não compila. Note que duas funções de rotas registram `/arquivos` com métodos diferentes (`GET` numa, `POST` noutra); o axum junta os métodos da mesma rota ao fazer `merge`, e a separação existe só para que cada grupo receba camadas diferentes.

5.3 Grupos com as camadas certas

Este é o ponto em que a lição do Módulo 2.2 (limites que algumas rotas afrouxam não podem ser globais) vira código:

```
// src/app.rs
fn rotas_v1(estado: AppState) -> Router<AppState> {
    let json = Router::new()
        .merge(arquivos::rotas_json())
        .merge(titulos::rotas())
        .layer(TimeoutLayer::new(Duration::from_secs(10))) // prazo curto
        .layer(DefaultBodyLimit::max(64 * 1024));           // corpos JSON
    pequenos

    let longas = arquivos::rotas_longas()
        .merge(ocorrencias::rotas_exportacao())
        .layer(DefaultBodyLimit::disable())                // o upload
    controla o próprio limite
        .layer(RequestBodyLimitLayer::new(500 * 1024 * 1024)); // teto de 500
    MiB no stream

    json.merge(longas)
        .route_layer(middleware::from_fn_with_state(estado.clone(),
    limites::por_cliente)) // 2. quantas vezes
        .route_layer(middleware::from_fn_with_state(estado,
    auth::autenticar))      // 1. quem é
    }
}
```

Três detalhes:

- **Ordem das camadas.** Como a última `.route_layer` chamada é a mais externa, a autenticação roda antes do rate limiting, que precisa do cliente identificado. Num `ServiceBuilder` a ordem é a de leitura; com chamadas encadeadas de `.layer`, é a inversa. Escolha um estilo por router e comente a ordem.
- **`route_layer` para autenticação.** Uma URL inexistente sob `/v1` dá 404, não 401.
- **`DefaultBodyLimit` x `RequestBodyLimitLayer`.** O primeiro limita os extractors que leem o corpo inteiro (`Json`, `Bytes`); o segundo limita o fluxo de bytes, inclusive o `Body` em streaming do upload.

5.4 Regras de acesso como extractors

O equivalente ao filtro `ExigirDonoDoArquivo` do .NET e ao middleware `exigirDono` do Go é, em Rust, um extractor cujo tipo só pode ser construído se a regra passou:

```
// src/arquivos/aceso.rs
pub struct ArquivoDoCliente {
    id: Uuid,
    cliente_id: Uuid,
} // campos privados: fora deste módulo, a única forma de obter um é pelo
extractor
```

```

impl ArquivoDoCliente {
    pub fn id(&self) -> Uuid { self.id }
    pub fn cliente_id(&self) -> Uuid { self.cliente_id }
}

impl FromRequestParts<AppState> for ArquivoDoCliente {
    type Rejection = ErroDaApi;

    async fn from_request_parts(parts: &mut Parts, estado: &AppState) ->
    Result<Self, Self::Rejection> {
        let Path(id) = Path::::from_request_parts(parts,
estado).await?;
        let usuario = Usuario::from_request_parts(parts, estado).await?;

        let pertence = sqlx::query_scalar!(
            "SELECT EXISTS (SELECT 1 FROM arquivo WHERE id = $1 AND
cliente_id = $2)",
            id, usuario.cliente_id
        )
        .fetch_one(&estado.banco)
        .await
        .map_err(anyhow::Error::from)?;

        if pertence != Some(true) {
            return Err(ErroDaApi::NaoEncontrado); // 404, nunca 403: não
confirme que o ID existe
        }
        Ok(Self { id, cliente_id: usuario.cliente_id })
    }
}

```

Compare com as outras duas linguagens. Lá, a regra era um filtro ou middleware **aplicado** à rota, e esquecer de aplicá-lo numa rota nova deixava a rota aberta. Aqui, uma função como `consulta::obter(&pool, &arquivo)` recebe um `&ArquivoDoCliente`, e a única forma de ter um é pelo extractor: um handler novo que esqueça a verificação não tem como chamar a consulta. A consulta ainda filtra pelo `cliente_id` (defesa em profundidade, como nos outros cursos), mas agora o tipo carrega o cliente junto com o ID.

5.5 Dependências: AppState, genéricos ou trait objects?

Há três formas de um handler chegar às suas dependências, e cada uma tem seu lugar:

Forma	Exemplo	Quando
Tipo concreto no <code>AppState</code>	<code>banco: PgPool</code>	Dependências que os testes de integração usam de verdade (o banco do container)
Trait object no <code>AppState</code>	<code>storage: Arc<dyn Storage></code>	Dependências externas trocadas nos testes (S3 por um storage em memória)
Genérico (<code>AppState<S: Storage></code>)	Despacho estático	Raramente compensa numa API: espalha o parâmetro de tipo por todo o código

Traits com métodos `async` não podem ser usados diretamente como `dyn` (o compilador não sabe o tamanho da future de cada implementação). Para um trait object como `Arc<dyn Storage>`, use o crate `async-trait`, que devolve futures em `Box`; o custo de uma alocação por chamada é irrelevante numa chamada ao S3. Onde não precisar de `dyn`, prefira o `async fn` nativo nos traits.

5.6 A composição

```
// src/app.rs
pub async fn montar_estado(config: &Config) -> anyhow::Result<AppState> {
    let banco = PgPoolOptions::new()
        .max_connections(20) // orçamento por réplica
    (Módulo 6)
        .acquire_timeout(Duration::from_secs(3))
        .connect(&config.banco_url).await?;
    Ok(AppState {
        banco,
        jwt: Arc::new(auth::Validador::novo(&config.jwt_emissor,
&config.jwt_audiencia).await?),
        storage: Arc::new(armazenamento::S3::novo(config).await?),
        limites: Arc::new(limites::PorCliente::novo(100)),
    })
}

pub fn montar_router(estados: AppState) -> Router {
    Router::new()
        .nest("/v1", rotas_v1(estados.clone()))
        .route("/health/live", get(saude::vivo))
        .route("/health/ready", get(saude::pronto))
}
```

```

        .fallback(erros::rota_inexistente)
        .layer(/* camadas globais do Módulo 2.2 */)
        .with_state(estado)
    }

```

Como no Go, o grafo de dependências está numa função que se lê de cima para baixo, verificada pelo compilador. As rotas de saúde ficam fora de `/v1` e, por isso, fora da autenticação e do rate limiting, sem precisar de exceção.

5.7 O que evitar

- **static global com o pool** (`LazyLock<PgPool>`): esconde a dependência e torna os testes dependentes de ordem. O pool vai no `AppState`.
- **Extension para dependências**: funciona, mas um `Extension` ausente só aparece como erro 500 em tempo de execução; `State` é verificado na compilação.
- **Um AppState com tudo público e mutável**: estado mutável compartilhado exige `Mutex` e vira gargalo; quase tudo no estado deve ser imutável ou internamente sincronizado (como o pool).
- **Genéricos em toda parte "por desempenho"**: numa API dominada por I/O, a diferença entre despacho estático e dinâmico não aparece no perfil, e o custo em legibilidade e tempo de compilação é real.

Exercício do Módulo 5

Organize o código no layout de 5.1, com `arquivos::rotas_json`, `arquivos::rotas_longas` e `titulos::rotas`, e monte o `rotas_v1` de 5.3. Implemente `ArquivoDoCliente` e use-o em `GET /v1/arquivos/{id}`. Teste que o arquivo de outro cliente devolve 404. Depois escreva um handler novo que tente chamar `consulta::obter` sem um `ArquivoDoCliente` e veja o compilador recusar. Por fim, coloque a `TimeoutLayer` de 10 segundos no router global e observe um upload grande ser cortado.

Módulo 6 — Dados: sqlx, paginação, streaming e upload

Numa API, o `sqlx` é a escolha natural em Rust: as macros `query!` e `query_as!` verificam cada consulta contra o schema real durante a compilação, o mesmo papel do `sqlc` no Go, sem gerar arquivos de código. As regras de desempenho são as de sempre: só as colunas necessárias, keyset em vez de `OFFSET`, streaming em vez de materializar, upload sem passar pela memória e `POST` idempotente.

6.1 O pool e as consultas verificadas

O `PgPool` do Módulo 5.6 é seguro para uso concorrente e barato de clonar. Para que as macros verifiquem as consultas, o compilador precisa do schema, de um de dois jeitos:

- **Com banco:** a variável `DATABASE_URL` aponta para um PostgreSQL com as migrations aplicadas (o do `docker-compose` de desenvolvimento).
- **Offline:** `cargo sqlx prepare` grava a descrição de cada consulta em `.sqlx/`, que vai para o repositório; o CI e o build da imagem compilam sem banco, e o comando `cargo sqlx prepare --check` falha se o cache estiver desatualizado.

As migrations são as do sistema inteiro (a tabela `arquivo` do curso de arquivos, Módulo 2.3, e as demais). Se os workers já as aplicam, a API só as lê: um único dono do schema.

6.2 Consultas: direto para o tipo de resposta

```
pub async fn obter(banco: &PgPool, arquivo: &ArquivoDoCliente) ->
Result<Option<ArquivoResposta>, ErroDaApi> {
    let resposta = sqlx::query_as!(
        ArquivoResposta,
        r#"SELECT id, nome_original, banco, status AS "status:
StatusArquivo", recebido_em,
                linhas_processadas, linhas_rejeitadas, motivo
        FROM arquivo
        WHERE id = $1 AND cliente_id = $2"#,
        arquivo.id(),
        arquivo.cliente_id(),
    )
    .fetch_optional(banco)
    .await
    .map_err(anyhow::Error::from)?;
    Ok(resposta)
}
```

A macro confere em compilação que cada coluna existe, que o tipo de cada uma corresponde ao campo de `ArquivoResposta` e que `motivo`, anulável no banco, é `Option<String>` na struct. A anotação `"status: StatusArquivo"` diz que a coluna de texto deve ser decodificada no `enum` do Módulo 3.3. Renomear uma coluna nas migrations quebra o build, não a produção. `fetch_optional` devolve `None` quando não há linha, sem um erro especial para tratar.

6.3 Paginação keyset com cursor opaco

O mesmo desenho das outras duas APIs: a coluna `sequencia`, o índice (`cliente_id`, `sequencia DESC`), pedir `limite + 1` linhas e um cursor opaco em Base64 URL-safe.

```

use base64::{engine::general_purpose::URL_SAFE_NO_PAD, Engine};

fn codificar_cursor(sequencia: i64) -> String {
    URL_SAFE_NO_PAD.encode(sequencia.to_be_bytes())
}

fn decodificar_cursor(cursor: Option<&str>) -> Result<i64, ErroDaApi> {
    let Some(c) = cursor else { return Ok(i64::MAX) }; // sem cursor: começa
do mais recente
    let bytes: [u8; 8] = URL_SAFE_NO_PAD
        .decode(c)
        .ok()
        .and_then(|b| b.try_into().ok())
        .ok_or_else(|| ErroDaApi::campo("cursor", "cursor inválido"))?;
    Ok(i64::from_be_bytes(bytes))
}

pub async fn listar(
    State(estados): State<AppState>,
    usuario: Usuario,
    ValidadoQuery(filtro): ValidadoQuery<FiltroDeArquivos>, // limite entre 1
e 200, de <= ate
) -> Result<Json<Pagina<ArquivoResumo>>, ErroDaApi> {
    let antes_de = decodificar_cursor(filtro.cursor.as_deref())?;
    let mut itens = sqlx::query_as!(
        ArquivoResumo,
        r#"SELECT id, nome_original, status AS "status: StatusArquivo",
recebido_em, sequencia
        FROM arquivo
        WHERE cliente_id = $1 AND sequencia < $2
        AND ($3::text IS NULL OR status = $3)
        AND ($4::date IS NULL OR data_recebimento >= $4)
        AND ($5::date IS NULL OR data_recebimento <= $5)
        ORDER BY sequencia DESC
        LIMIT $6"#,
        usuario.cliente_id, antes_de,
        filtro.status.map(StatusArquivo::como_texto), filtro.de, filtro.ate,
        i64::from(filtro.limite) + 1,
    )
    .fetch_all(&estados.banco)
    .await
    .map_err(anyhow::Error::from)?;

    let tem_mais = itens.len() > filtro.limite as usize;
    itens.truncate(filtro.limite as usize);
}

```

```

    let proximo_cursor = tem_mais.then(||
codificar_cursor(itens.last().unwrap().sequencia));
    Ok(Json(Pagina { itens, proximo_cursor }))
}

```

O `ArquivoResumo` tem `#[serde(skip)]` na `sequencia`: ela existe só para montar o cursor e não vaza para o contrato. Os filtros opcionais viram uma consulta só, com o padrão `($3::text IS NULL OR status = $3)`, e `ErroDaApi::campo` é um atalho que cria um `Validacao` com um único campo.

6.4 Respostas em streaming

Para a exportação em CSV, o corpo da resposta é um `Stream` de blocos de bytes. O `sqlx` já entrega as linhas como stream (`fetch`), mas esse stream empresta o pool, e o corpo de uma resposta precisa ser `'static`. A macro `try_stream!` do crate `async-stream` resolve, movendo um clone do pool para dentro do gerador:

```

pub async fn exportar(
    State(estado): State<AppState>,
    usuario: Usuario,
    Query(ExportacaoQuery { dia }): Query<ExportacaoQuery>,
) -> impl IntoResponse {
    let banco = estado.banco.clone();
    let linhas = async_stream::try_stream! {
        yield
Bytes::from_static(b"nosso_numero;ocorrencia;data;valor_centavos\n");
        let mut cursor = sqlx::query!(
            "SELECT d.nosso_numero, d.codigo_ocorrencia, d.data_ocorrencia,
d.valor_pago_centavos
            FROM retorno_detalhe d JOIN arquivo a ON a.id = d.arquivo_id
            WHERE a.cliente_id = $1 AND d.data_ocorrencia = $2",
            usuario.cliente_id, dia,
        )
        .fetch(&banco);
        while let Some(l) = cursor.try_next().await? {
            yield Bytes::from(format!("{}", l.nosso_numero, l.codigo_ocorrencia, l.data_ocorrencia,
l.valor_pago_centavos));
        }
    };
    (
        [
            (header::CONTENT_TYPE, "text/csv; charset=utf-8".to_string()),
            (header::CONTENT_DISPOSITION, format!("attachment;
filename=\"ocorrencias-{}csv\"", dia)),

```

```

    ],
    Body::from_stream(linhas),
)
}

```

O problema que os outros cursos resolviam à mão (um erro no meio de uma resposta que já começou com 200) aqui é tratado pelo tipo: quando o stream produz `Err`, o hyper interrompe o corpo e fecha a conexão sem terminá-lo, e o cliente vê erro de transferência, não um CSV truncado que parece completo. É o que o Go fazia com

`panic(http.ErrAbortHandler)`. Um `format!` por linha aloca, o que é aceitável numa exportação limitada pela rede; para volumes enormes, o `copy_out_raw` do `sqlx` faz o PostgreSQL gerar o CSV e devolve os blocos prontos, como no curso de arquivos em Rust.

6.5 Upload de arquivos grandes

O extractor `Body` entrega o corpo em streaming, e `into_data_stream()` o transforma num stream de blocos. O serviço de recebimento calcula o hash e envia ao S3 em partes enquanto lê, sem que o arquivo exista inteiro na memória:

```

const PARTE: usize = 8 * 1024 * 1024; // o upload multipart do S3 exige
partes de pelo menos 5 MiB

pub async fn receber(estado: &AppState, pedido: PedidoDeRecebimento, corpo:
Body)
-> Result<ArquivoResposta, ErroDaApi>
{
    let id = Uuid::now_v7();
    let chave = format!("retornos/{}/{id}", Utc::now().format("%Y/%m/%d"));

    let mut hash = Sha256::new();
    let mut envio = estado.storage.iniciar_envio(&chave).await?; // upload
multipart
    let mut parte = BytesMut::with_capacity(PARTE);
    let mut fluxo = corpo.into_data_stream();

    while let Some(bloco) = fluxo.try_next().await.map_err(erros::do_corpo)?
    {
        hash.update(&bloco);
        parte.extend_from_slice(&bloco);
        if parte.len() >= PARTE {
            envio.enviar_parte(parte.split().freeze()).await?; // a memória
fica em torno de uma parte
        }
    }
    if !parte.is_empty() {

```

```

        envio.enviar_parte(parte.freeze()).await?;
    }
    envio.concluir().await?;
    let sha256 = hex::encode(hash.finalize());

    let mut tx = estado.banco.begin().await.map_err(anyhow::Error::from)?;
    let criado = sqlx::query_as!(ArquivoResposta, /* INSERT ... ON CONFLICT
(cliente_id, sha256) DO NOTHING RETURNING ... */)
        .fetch_optional(&mut *tx).await.map_err(anyhow::Error::from)?;
    let Some(criado) = criado else {
        let existente = sqlx::query_scalar!("SELECT id FROM arquivo WHERE
cliente_id = $1 AND sha256 = $2",
            pedido.cliente_id, sha256).fetch_one(&mut
*tx).await.map_err(anyhow::Error::from)?;
        estado.storage.remove(&chave).await.ok(); // duplicado: descarta o
objeto recém-enviado
        return Err(ErroDaApi::ArquivoDuplicado { existente });
    };
    sqlx::query!(/* INSERT INTO outbox ... evento arquivo.recebido
*/).execute(&mut *tx).await.map_err(anyhow::Error::from)?;
    tx.commit().await.map_err(anyhow::Error::from)?; // sem commit, o Drop da
transação faz rollback
    Ok(criado)
}

```

Pontos específicos de Rust:

- **O limite de 500 MiB** vem do `RequestBodyLimitLayer` do Módulo 5.3. Quando o corpo passa dele, o stream devolve um erro cuja causa é um `LengthLimitError`; a função `erros::do_corpo` percorre a cadeia de causas (`std::error::Error::source`) e devolve `CorpoGrandeDemais` (413) nesse caso.
- **Cancelamento por Drop**. Se o cliente desconectar no meio, a future do handler é descartada e o código depois do `.await` atual simplesmente não roda: nenhuma limpeza, nenhum "abortar upload". Diferente do `context.WithoutCancel` do Go, não há como continuar a limpeza no mesmo fluxo. Duas defesas: uma regra de ciclo de vida no bucket que aborta uploads multipart incompletos após um dia, e um `Drop` no tipo `envio` que, se ele não foi concluído, dispara a chamada de abortar numa tarefa separada (`tokio::spawn`).
- **Clientes lentos**. O `RequestBodyTimeoutLayer` do `tower-http` encerra a requisição se o corpo ficar parado por tempo demais, sem limitar a duração total de um upload grande que está progredindo.

- O **handler** fica pequeno: confere `Content-Type: application/octet-stream`, monta o `PedidoDeRecebimento` (validado) a partir da query e do cabeçalho `Idempotency-Key`, chama `receber` e devolve `(StatusCode::ACCEPTED, [(LOCATION, ...)], Json(arquivo))`.

6.6 Idempotência no POST

O algoritmo é o do curso de .NET (Módulo 6.6, com o diagrama de decisão). Em axum, ele cabe num middleware aplicado só à rota de upload, que guarda a resposta depois que o handler a produz:

```
async fn idempotencia(State(estado): State<AppState>, usuario: Usuario, req:
Request, next: Next)
    -> Result<Response, ErroDaApi>
{
    let Some(chave) = req.headers().get("idempotency-key").and_then(|v|
v.to_str().ok()).map(str::to_owned) else {
        return Ok(next.run(req).await); // sem chave: segue normalmente
    };
    match registrar_chave(&estado.banco, usuario.cliente_id, &chave,
hash_do_pedido(&req)).await? {
        Chave::Nova => {}
        Chave::Concluida(gravada) => return Ok(gravada.into_response()), //
repetição: devolve o que já foi respondido
        Chave::EmAndamento => return Err(ErroDaApi::EmAndamento), //
409
        Chave::OutroConteudo => return Err(ErroDaApi::ChaveReutilizada), //
422
    }
    let resposta = next.run(req).await;
    let (partes, corpo) = resposta.into_parts();
    let bytes = axum::body::to_bytes(corpo, 64 *
1024).await.map_err(anyhow::Error::from)?; // respostas pequenas
    gravar_resposta(&estado.banco, usuario.cliente_id, &chave, partes.status,
&partes.headers, &bytes).await?;
    Ok(Response::from_parts(partes, Body::from(bytes)))
}
```

O `enum Chave` deixa explícitos os quatro desfechos do diagrama, e o `match` obriga a tratar cada um. As duas novas variantes de `ErroDaApi` (`EmAndamento`, `ChaveReutilizada`) entram no mapeamento central, e o compilador lembra disso.

6.7 Concorrência otimista

Como nas outras APIs: `xmin` como versão, exposta como `ETag` no `GET`, exigida como `If-Match` no `PUT`. A atualização inclui `AND xmin::text = $versao`; se `rows_affected()` for zero, uma consulta de existência decide entre 404 e 412 Precondition Failed.

Exercício do Módulo 6

Configure o `sqlx` com as migrations, gere o cache com `cargo sqlx prepare` e confirme que o projeto compila sem banco. Renomeie uma coluna numa migration e veja a consulta parar de compilar. Implemente `GET /v1/arquivos` com cursor e confira com `EXPLAIN` que a página 100 custa o mesmo que a primeira. Envie 400 MB com `curl --data-binary` e acompanhe a memória do processo. Por fim, derrube o banco no meio de uma exportação e confirme que o cliente recebe erro de transferência, não um CSV truncado.

Módulo 7 — Contrato: OpenAPI e versionamento

Em Rust, a forma mais comum de ter um documento OpenAPI é **gerá-lo dos tipos**, como no .NET: o crate `utoipa` deriva schemas das structs de entrada e saída e lê anotações nos handlers. A diferença importante em relação ao .NET é que parte da anotação (as respostas possíveis de cada rota) é declarada ao lado do handler e **não é conferida** contra o tipo que ele devolve. Este módulo usa o `utoipa` e fecha essa lacuna com testes de contrato.

7.1 As abordagens em Rust

Abordagem	Como funciona	Prós	Contras
<code>utoipa</code> + <code>utoipa-axum</code>	<code>derive(ToSchema)</code> nos tipos, <code>#[utoipa::path]</code> nos handlers	O mais usado; maduro; schemas fiéis aos tipos	Respostas declaradas à mão podem divergir do handler
<code>aide</code>	Gera o documento a partir dos extractors e tipos de retorno do axum	Menos duplicação	Menos difundido; exige handlers no formato dele
Spec-first (openapi-generator <code>rust-axum</code>)	<code>openapi.yaml</code> → traits e tipos gerados	O documento é a fonte da verdade, como no curso de Go	Código gerado verboso; menos idiomático
<code>poem-openapi</code>	OpenAPI de primeira classe	Contrato e código inseparáveis	Exige trocar o framework

7.2 Schemas a partir dos tipos

```
#[derive(Debug, Serialize, utoipa::ToSchema)]
#[serde(rename_all = "camelCase")]
/// Situação de um arquivo de retorno recebido.
pub struct ArquivoResposta {
    /// Identificador do arquivo, devolvido no recebimento.
    pub id: Uuid,
    pub nome_original: String,
    pub banco: String,
    pub status: StatusArquivo,
    pub recebido_em: DateTime<Utc>,
    pub linhas_processadas: i32,
    /// Linhas de detalhe que falharam na validação do layout.
    pub linhas_rejeitadas: i32,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub motivo: Option<String>,
}
```

O `ToSchema` lê os atributos do `serde` (o `rename_all`, o `skip`, os `Option`), então o schema publicado usa os mesmos nomes que o JSON real. Os comentários de documentação (`///`) viram descrições no documento, como os comentários XML do

.NET 10. O mesmo derive vai em `StatusArquivo` (vira um `enum` de strings), em `ProblemDetails` e nos tipos de entrada; `IntoParams` faz o mesmo para as structs de `Query`.

7.3 Rotas anotadas

```
#[utoipa::path(
    get,
    path = "/arquivos/{id}",
    tag = "Arquivos",
    params(("id" = Uuid, Path, description = "Identificador do arquivo")),
    responses(
        (status = 200, description = "Arquivo encontrado", body =
ArquivoResposta),
        (status = 404, description = "Inexistente ou de outro cliente",
        body = ProblemDetails, content_type = "application/problem+json"),
    ),
    security(("bearer" = [])),
)]
pub async fn obter(State(estado): State<AppState>, arquivo: ArquivoDoCliente)
    -> Result<Json<ArquivoResposta>, ErroDaApi> { /* ... */ }

// src/arquivos/mod.rs: o caminho registrado no router é o da anotação
pub fn rotas_json() -> OpenApiRouter<AppState> {
    OpenApiRouter::new()
        .routes(routes!(handlers::listar))
        .routes(routes!(handlers::obter))
}
```

O `OpenApiRouter` e a macro `routes!` do `utoipa-axum` registram a rota usando o método e o caminho da própria anotação, então caminho e método nunca divergem. No topo, `OpenApiRouter::split_for_parts()` separa o `Router` normal do documento coletado de todas as features.

O que **não** é conferido: se `obter` passar a devolver também `ErroDaApi::Proibido` (403), nada obriga a anotação a listar o 403. É a mesma classe de risco do swaggo no Go, mais contida (só as respostas, não os tipos). A seção 7.5 fecha a lacuna nos testes.

7.4 Segurança, documento e interface

O esquema `bearer` é declarado uma vez, com o trait `Modify` do `utoipa`:

```

#[derive(utoipa::OpenApi)]
#[openapi(info(title = "API de Cobrança", version = "v1"),
modifiers(&SegurancaBearer))]
pub struct DocumentoV1;

struct SegurancaBearer;
impl utoipa::Modify for SegurancaBearer {
    fn modify(&self, doc: &mut utoipa::openapi::OpenApi) {
        let componentes =
doc.components.get_or_insert_with(Default::default);
        componentes.add_security_scheme(
            "bearer",

SecurityScheme::Http(HttpBuilder::new().scheme(HttpAuthScheme::Bearer).bearer
_format("JWT").build()),
        );
    }
}

```

O documento é servido em `/openapi.json` por uma rota comum, e o crate `utoipa-scalar` monta a interface do Scalar sobre ele (só em desenvolvimento, ou atrás de autenticação, pelas mesmas razões dos outros cursos).

7.5 O contrato no CI

Três camadas de proteção:

1. **Snapshot do documento.** Um teste gera o JSON do OpenAPI e o compara com um arquivo aprovado no repositório, usando o crate `insta`. Toda mudança de contrato aparece no pull request como diff do snapshot e precisa ser aprovada (cargo `insta review`).
2. **Quebras detectadas.** O `oasdiff` compara o documento do pull request com o da branch principal e falha em mudanças incompatíveis, como nos outros cursos.
3. **Respostas reais conferidas.** Nos testes de integração (Módulo 10), cada resposta é validada contra o schema e os status declarados para aquela rota no documento gerado. Se um handler devolver um 403 que a anotação não lista, o teste falha. É isso que fecha a lacuna de 7.3.

7.6 Versionamento

As regras do curso de .NET (Módulos 7.7 e 7.8) valem integralmente: versão no caminho, nova versão só para mudanças incompatíveis, cuidado com valores novos em `enums`, `Deprecation` e `Sunset` para aposentar. Em axum, cada versão é um `nest` com o seu `OpenApiRouter` e o seu documento:

```
let (v1, doc_v1) = rotas_v1(estado.clone()).split_for_parts();
let (v2, doc_v2) = rotas_v2(estado.clone()).split_for_parts();
Router::new()
    .nest("/v1", v1)
    .nest("/v2", v2) // só o que mudou ganha handler novo; o resto
    reaproveita os da v1
    .route("/openapi/v1.json", get(move || async move { Json(doc_v1) })))
    .route("/openapi/v2.json", get(move || async move { Json(doc_v2) })))
```

Com `enum`s, há um cuidado específico de Rust: se a API consome `enum`s vindos de outros sistemas (como eventos), use `#[serde(other)]` numa variante `Desconhecido` para que um valor novo não quebre a desserialização. É o mesmo conselho que a API dá aos seus clientes, aplicado a ela mesma.

Exercício do Módulo 7

Derive `ToSchema` nos contratos e `IntoParams` nos filtros, anote `obter` e `listar` e sirva o documento com o Scalar. Escreva o teste de snapshot com `insta` e confirme que renomear um campo aparece no diff. Depois faça `obter` devolver `ErroDaApi::Proibido` num caso proposital sem atualizar a anotação: o build passa, e o teste de contrato do Módulo 10 é quem deve pegar. Por fim, configure o `oasdiff` no CI.

Módulo 8 — Segurança: quem pode chamar o quê, e quantas vezes

As camadas são as das outras duas APIs: autenticação por JWT de um provedor externo, autorização por escopo, autorização por recurso e limites por cliente. Em Rust, as três primeiras podem aparecer na **assinatura do handler**: um parâmetro

`ComEscopo<EscritaDeArquivos>` diz, e o compilador garante, que o handler só roda para um usuário autenticado com aquele escopo; um `ArquivoDoCliente` (Módulo 5.4) garante a posse do recurso.

8.1 Validando o JWT com as chaves do provedor

O crate `jsonwebtoken` valida assinatura, emissor, audiência e validade. As chaves públicas vêm do JWKS do provedor, descoberto pela configuração OpenID, e ficam em cache, com recarga quando chega um token assinado por uma chave nova (rotação de chaves):

```
use jsonwebtoken::{decode, decode_header, jwk::JwkSet, DecodingKey,
Validation, Algorithm};

#[derive(Debug, Deserialize)]
struct Claims {
```

```

    sub: String,
    cliente_id: Uuid,
    #[serde(default)]
    scope: String, // escopos separados por espaço
}

pub struct Validador {
    chaves: tokio::sync::RwLock<JwkSet>,
    url_jwks: String,
    validacao: Validation,
    http: reqwest::Client,
}

impl Validador {
    pub async fn novo(emissor: &str, audiencia: &str) -> anyhow::Result<Self>
    {
        let http = reqwest::Client::new();
        let descoberta: serde_json::Value = http
            .get(format!("{emissor}/.well-known/openid-configuration"))
            .send().await?.json().await?;
        let url_jwks = descoberta["jwks_uri"].as_str().context("jwks_uri ausente")?.to_owned();
        let chaves = http.get(&url_jwks).send().await?.json:::<JwkSet>().await?;

        let mut validacao = Validation::new(Algorithm::RS256);
        validacao.set_issuer(&[emissor]);
        validacao.set_audience(&[audiencia]); // obrigatório: sem ela, tokens de outras APIs passariam
        validacao.leeway = 30; // tolerância de relógio, em segundos

        Ok(Self { chaves: tokio::sync::RwLock::new(chaves), url_jwks, validacao, http })
    }

    pub async fn validar(&self, token: &str) -> Result<Identidade, ErroDaApi>
    {
        let kid = decode_header(token).ok().and_then(|h| h.kid).ok_or(ErroDaApi::NaoAutenticado)?;
        let chave = match self.chave(&kid).await {
            Some(c) => c,
            None => {
                self.recarregar().await; // chave desconhecida: talvez o provedor tenha rotacionado
                self.chave(&kid).await.ok_or(ErroDaApi::NaoAutenticado)?
            }
        }
    }
}

```

```

    }
};
let dados = decode::<Claims>(token, &chave,
&self.validacao).map_err(|e| {
    tracing::info!(erro = %e, "token recusado"); // o motivo só no
log
    ErroDaApi::NaoAutenticado
})?;
Ok(Identidade {
    sujeito: dados.claims.sub,
    cliente_id: dados.claims.cliente_id,
    escopos:
dados.claims.scope.split_whitespace().map(str::to_owned).collect(),
})
}

async fn chave(&self, kid: &str) -> Option<DecodingKey> {
    let chaves = self.chaves.read().await;
    chaves.find(kid).and_then(|jwk| DecodingKey::from_jwk(jwk).ok())
}

async fn recarregar(&self) {
    if let Ok(novas) = async {
self.http.get(&self.url_jwks).send().await?.json::<JwkSet>().await }.await {
    *self.chaves.write().await = novas;
}
}
}
}

```

O mesmo cuidado dos outros cursos com a audiência, e o mesmo tratamento dos escopos (`split_whitespace`, porque a claim `scope` vem separada por espaços). A claim `cliente_id` é desserializada direto como `Uuid`: um token sem ela, ou com um valor que não é UUID, falha na desserialização e vira 401. Em produção, limite a frequência de `recarregar` (por exemplo, no máximo uma vez por minuto), para que tokens com `kid` inventado não virem uma enxurrada de chamadas ao provedor.

8.2 O middleware e os extractors de identidade

O middleware do grupo `/v1` (Módulo 5.3) valida o token e guarda a identidade nas extensões da requisição; os extractors a leem:

```

pub async fn autenticar(State(estado): State<AppState>, mut req: Request,
next: Next)
-> Result<Response, ErroDaApi>
{

```

```

    let token = req.headers().get(header::AUTHORIZATION)
        .and_then(|v| v.to_str().ok())
        .and_then(|v| v.strip_prefix("Bearer "))
        .ok_or(ErroDaApi::NaoAutenticado)?;
    let identidade = estado.jwt.validar(token).await?;
    req.extensions_mut().insert(identidade);
    Ok(next.run(req).await)
}

/// A identidade autenticada. Usar numa rota fora do grupo autenticado é bug:
vira 500 registrado.
#[derive(Debug, Clone)]
pub struct Usuario(pub Identidade);

// Deref: usuario.cliente_id funciona direto, sem usuario.0
impl std::ops::Deref for Usuario {
    type Target = Identidade;
    fn deref(&self) -> &Identidade { &self.0 }
}

impl<S: Send + Sync> FromRequestParts<S> for Usuario {
    type Rejection = ErroDaApi;
    async fn from_request_parts(parts: &mut Parts, _: &S) -> Result<Self,
Self::Rejection> {
        parts.extensions.get:::<Identidade>().cloned().map(Usuario)
            .ok_or_else(|| ErroDaApi::Interno(anyhow::anyhow!("rota sem o
middleware de autenticação"))))
    }
}

```

A escolha do 500 (e não 401) para identidade ausente é a mesma do `panic` em `auth.Usuario` do curso de Go: uma rota de negócio registrada fora do grupo autenticado é um erro de programação e deve aparecer alto, não como "não autenticado". Para que o `ErroDaApi::NaoAutenticado` saia com `WWW-Authenticate: Bearer`, o `IntoResponse` do Módulo 4 já acrescenta o cabeçalho.

8.3 Escopos no tipo do parâmetro

Um extractor genérico sobre um tipo marcador transforma o escopo exigido em parte da assinatura:

```

pub trait Escopo { const NOME: &'static str; }

pub struct EscritaDeArquivos;
impl Escopo for EscritaDeArquivos { const NOME: &'static str =
"arquivos:escrever"; }

```

```

pub struct LeituraDeCobranca;
impl Escopo for LeituraDeCobranca { const NOME: &'static str =
"cobranca:ler"; }

pub struct ComEscopo<E: Escopo>(pub Usuario, PhantomData<E>);

impl<E: Escopo, S: Send + Sync> FromRequestParts<S> for ComEscopo<E> {
    type Rejection = ErroDaApi;
    async fn from_request_parts(parts: &mut Parts, estado: &S) ->
Result<Self, Self::Rejection> {
        let usuario = Usuario::from_request_parts(parts, estado).await?;
        if !usuario.0.escopos.iter().any(|e| e == E::NOME) {
            return Err(ErroDaApi::Proibido); // 403
        }
        Ok(Self(usuario, PhantomData))
    }
}

// a assinatura documenta e o compilador impõe
pub async fn receber(
    State(estado): State<AppState>,
    ComEscopo(usuario, _): ComEscopo<EscritaDeArquivos>,
    headers: HeaderMap,
    corpo: Body,
) -> Result<impl IntoResponse, ErroDaApi> { /* ... */ }

```

Quem lê o handler sabe exatamente o que ele exige, sem procurar o `RequireAuthorization` na configuração de rotas. E é impossível o handler rodar sem a verificação: o extractor é o único jeito de obter o `ComEscopo`.

8.4 Autorização por recurso

A primeira falha da lista OWASP API Security Top 10 (*Broken Object Level Authorization*) é atacada pelas mesmas camadas: o extractor `ArquivoDoCliente` do Módulo 5.4, o filtro por cliente em toda consulta (que o `sqlx` verifica existir na consulta, mas não obriga) e, opcionalmente, Row Level Security no PostgreSQL, configurando `app.cliente_id` no início de cada transação, como descrito no curso de Go. Objeto de outro cliente responde 404.

8.5 Rate limiting por cliente

O crate `governor` implementa o algoritmo GCRA (equivalente a um token bucket com reposição contínua) com um limitador por chave:

```

use governor::{DefaultKeyedRateLimiter, Quota};
use std::num::NonZeroU32;

pub struct PorCliente(DefaultKeyedRateLimiter<Uuid>);

impl PorCliente {
    pub fn novo(por_minuto: u32) -> Self {
        // rajada de por_minuto e uma ficha reposta a cada 60/por_minuto
        segundos

        Self(governor::RateLimiter::keyed(Quota::per_minute(NonZeroU32::new(por_minuto).unwrap()))
        )
    }
}

pub async fn por_cliente(State(estado): State<AppState>, usuario: Usuario,
req: Request, next: Next)
-> Result<Response, ErroDaApi>
{
    if let Err(negado) = estado.limite.check_key(&usuario.cliente_id) {
        let espera =
negado.wait_time_from(governor::clock::Clock::now(&governor::clock::DefaultClock::default()));
        return Err(ErroDaApi::LimiteExcedido { tentar_em_segundos:
espera.as_secs().max(1) });
    }
    Ok(next.run(req).await)
}

```

Três observações, as mesmas dos outros cursos com um detalhe a mais. A reposição é contínua, como no `rate.Limiter` do Go. O limitador vive na memória de cada réplica, então o limite efetivo multiplica pelo número de réplicas; limites contratuais exatos ficam no gateway ou num armazenamento compartilhado. E o mapa de chaves cresce com o número de clientes: chame `retain_recent()` periodicamente (uma tarefa com `tokio::time::interval`) para remover clientes inativos. O crate `tower_governor` empacota a mesma lógica como camada `tower`, mas sua chave padrão é o IP, não o cliente.

Para limitar uploads simultâneos por cliente, um `tokio::sync::Semaphore` por cliente (num `DashMap<Uuid, Arc<Semaphore>>`), com `try_acquire_owned()` no início do handler de upload: se não há permissão, 429; se há, a permissão é devolvida pelo `Drop` quando o handler termina, mesmo que ele termine por erro.

8.6 CORS e o resto da lista

Se um portal web chama a API do navegador, o `CorsLayer` do `tower-http` vai na pilha global, com origens, métodos e cabeçalhos explícitos (e `expose_headers` com `Location` e `Retry-After`), antes da autenticação, porque *preflights* não trazem token. O resto da lista é o dos outros cursos, com as ferramentas de Rust: TLS no ingress (ou `rustls`, se a API terminar TLS), limites de corpo por grupo de rotas (Módulo 5.3), segredos lidos uma vez na configuração, nada de token ou dado pessoal nos logs (o `TraceLayer` não registra cabeçalhos por padrão; mantenha assim) e `cargo audit` ou `cargo deny` no CI contra a base de vulnerabilidades RustSec.

Exercício do Módulo 8

Suba o Keycloak com dois clientes de máquina e escopos diferentes, como nos outros cursos. Implemente o `Validador`, o middleware `autenticar`, `Usuario`, `ComEscopo` e o `por_cliente`. Teste: sem token (401 com `WWW-Authenticate`), audiência errada (401), escopo insuficiente (403), arquivo de outro cliente (404) e 101 chamadas rápidas (429 com `Retry-After`). Rotacione a chave de assinatura no Keycloak e confirme que tokens novos continuam funcionando sem reiniciar a API. Por fim, registre uma rota que usa `Usuario` fora do grupo autenticado e veja o 500 com a mensagem no log.

Módulo 9 — Performance sob carga

As alavancas são as das outras duas APIs, na mesma ordem: não repetir trabalho (cache com proteção contra stampede), não esperar além do necessário (prazos), proteger-se de dependências lentas e só então otimizar o processo. Rust não tem GC nem JIT para aquecer, e o footprint de memória é pequeno; os problemas de desempenho que sobram são quase todos de I/O, de bloqueio do runtime e de contenção.

9.1 Medir primeiro

O número que importa continua sendo o p95/p99 sob carga realista, medido com `k6`, `NBomber` ou `oha` (uma ferramenta de carga escrita em Rust). Para enxergar dentro do processo, as ferramentas do curso de arquivos em Rust (Módulo 9.4): `samply` ou `cargo flamegraph` para CPU, `tokio-console` para tarefas que seguram a thread entre dois `.await`, e o `TraceLayer` com o histograma de duração por rota (Módulo 11).

9.2 Prazos: do servidor até o banco

- **Por grupo de rotas:** o `TimeoutLayer` de 10 segundos nas rotas JSON (Módulo 5.3). Quando ele estoura, a future do handler é descartada e o cliente recebe a resposta da camada; mapeie-a para `ProblemDetails` com 504.

- **No banco:** o `acquire_timeout` do pool (Módulo 5.6) limita a espera por uma conexão livre, e um `statement_timeout` configurado nas opções de conexão (`PgConnectOptions::options([("statement_timeout", "10s")])`) faz o servidor abortar consultas longas. Este segundo é indispensável em Rust: como visto no curso de arquivos (Módulo 7.6), descartar a future de uma consulta não cancela a consulta no PostgreSQL.
- **Nas chamadas de saída:** um `request::Client` criado uma vez, compartilhado pelo `AppState`, com `timeout` e `connect_timeout` definidos. O `Client` mantém um pool de conexões por host; criar um por requisição descarta esse pool. Para retry com backoff e jitter, os crates `request-middleware` e `request-retry`; para circuit breaker, as camadas do próprio `tower` funcionam também do lado cliente.

9.3 Cache com proteção contra stampede

O crate `moka` oferece um cache assíncrono com limite de tamanho, expiração e, o mais importante aqui, **carregamento coalescido**: várias requisições pedindo a mesma chave ao mesmo tempo esperam uma única execução da função de carga. É o `singleflight` do Go e a proteção do `HybridCache` do .NET, num tipo só:

```
pub type CacheDeTitulos = moka::future::Cache<Uuid, NossoNumero>,
Option<TituloResposta>>;

pub fn novo_cache() -> CacheDeTitulos {
    moka::future::Cache::builder()
        .max_capacity(100_000)                // limite de tamanho: nada
de crescer sem fim
        .time_to_live(Duration::from_secs(60))
        .build()
}

pub async fn obter(estado: &AppState, cliente: Uuid, numero: NossoNumero)
-> Result<Option<TituloResposta>, ErroDaApi>
{
    let banco = estado.banco.clone();
    estado.titulos
        .try_get_with((cliente, numero), async move { // uma carga por chave,
mesmo com 100 pedidos juntos
            sqlx::query_as!(TituloResposta,
                "SELECT nosso_numero, situacao, valor_centavos, vencimento
                FROM titulo WHERE cliente_id = $1 AND nosso_numero = $2",
                cliente, numero.valor())
                .fetch_optional(&banco)
                .await
        })
}
```

```

        .await
        .map_err(|e| ErroDaApi::Interno(anyhow::anyhow!(e))) // o erro chega
        como Arc<sqlx::Error>
    }

```

O cliente faz parte da chave, como nos outros cursos. Um título inexistente (`None`) também fica em cache, o que protege o banco de buscas repetidas por números que não existem e exige invalidar a chave quando o título é criado. Erros não são guardados:

`try_get_with` só armazena o resultado de sucesso. A invalidação por evento (quando o pipeline atualiza um título) usa `invalidate(&chave)` ; para um nível compartilhado entre réplicas, coloque um Redis atrás da função de carga.

9.4 Server-Sent Events

O axum tem SSE nativo: um handler devolve `Sse<impl Stream<Item = Result<Event, E>>>`, e o tipo `KeepAlive` envia comentários periódicos para manter a conexão viva em proxies, o que nos outros cursos era feito à mão:

```

pub async fn progresso(
    State(estado): State<AppState>,
    arquivo: ArquivoDoCliente, // a posse foi verificada antes do stream
    começar
) -> Sse<impl Stream<Item = Result<Event, Infallible>>> {
    let (banco, desligando) = (estado.banco.clone(),
estado.desligando.clone());
    let eventos = async_stream::stream! {
        let mut anterior = None;
        let mut intervalo = tokio::time::interval(Duration::from_secs(1));
        loop {
            tokio::select! {
                _ = intervalo.tick() => {}
                _ = desligando.cancelled() => break, // o servidor está
desligando (Módulo 11)
            }
            let Ok(Some(atual)) = consulta::progresso(&banco, &arquivo).await
            else { break };
            if anterior.as_ref() != Some(&atual) {
                yield
                Ok(Event::default().event("progresso").json_data(&atual).unwrap());
                anterior = Some(atual.clone());
            }
            if atual.status.final_() {
                break; // estado final: encerra o stream
            }
        }
    }
}

```

```

    }
};
Sse::new(eventos).keep_alive(KeepAlive::default())
}

```

Pontos de produção:

- **O cliente fechar a conexão** descarta o stream e, com ele, o laço: não há código de cancelamento para escrever.
- **O servidor desligar** não encerra o stream sozinho: o desligamento gracioso do axum espera as requisições em andamento, e um stream infinito seguraria o desligamento. Por isso o `desligando` (um `CancellationToken` no `AppState`, cancelado no início do desligamento, Módulo 11) entra no `select!`.
- **Compressão:** o predicado padrão do `CompressionLayer` não comprime `text/event-stream`, o que é o certo, porque a compressão seguraria os eventos num buffer.
- **Escala:** consultar o banco a cada segundo por conexão serve para dezenas de conexões; para milhares, troque por notificações (`LISTEN/NOTIFY` com o `PgListener` do `sqlx`, ou `Redis`), com um `tokio::sync::broadcast` distribuindo para os streams.

9.5 Compressão

O `CompressionLayer` do `tower-http` negocia gzip, Brotli e zstd pelo `Accept-Encoding`, ignora respostas pequenas e tipos já comprimidos e respeita streaming. Se o ingress já comprime, remova a camada; e as ressalvas de BREACH dos outros cursos continuam valendo.

9.6 As causas mais comuns de lentidão numa API em Rust

- **Trabalho bloqueante no handler:** hash de senha, compressão grande, `std::fs`, serialização de um JSON enorme. Tudo isso para uma thread do runtime; use `tokio::task::spawn_blocking`.
- **Segurar uma conexão do pool através de esperas longas** (uma chamada HTTP de saída no meio de uma transação): o pool esgota e todas as requisições esperam o `acquire_timeout`.
- **Contenção num `Mutex`** do estado compartilhado: um `Mutex<HashMap>` como cache vira o gargalo da API inteira. Use estruturas concorrentes (`moka`, `DashMap`) ou desenhe sem estado mutável.
- **N+1 consultas, e `clone()` e `format!` em laços longos** de serialização, visíveis num flame graph em minutos.

- **Esquecer o perfil de release:** um binário de debug é de dez a cem vezes mais lento. Benchmarks e testes de carga sempre com `--release` e o perfil do curso de arquivos em Rust (Módulo 9.6).

Exercício do Módulo 9

Rode um teste de carga de 2 minutos em `GET /v1/titulos/{nosso_numero}` com `oha` e anote o p95; adicione o `CacheDeTitulos` e repita. Simule um banco lento com `pg_sleep` e confirme que o `TimeoutLayer` responde 504 e que o `statement_timeout` aborta a consulta no servidor (confira em `pg_stat_activity`). Abra o stream de progresso no navegador com `EventSource` enquanto um worker processa um arquivo, e depois mande `SIGTERM` para a API e confirme que o stream é encerrado e o desligamento termina.

Módulo 10 — Testes: provar que tudo funciona

Um `Router` do axum é um `tower::Service`, então testar a API inteira não exige servidor nem porta: o método `oneshot` envia uma requisição ao router e devolve a resposta, passando por todas as camadas, extractors e handlers reais. Somado a um banco PostgreSQL de verdade, isolado por teste com `#[sqlx::test]`, e a tokens JWT reais assinados por uma chave de teste, isso dá testes de integração com a fidelidade do `WebApplicationFactory` do .NET e a velocidade do `httptest` do Go.

10.1 Unidade e propriedades

Tipos de domínio e funções puras (o `NossoNumero`, os cursores, o mapeamento de `ErroDaApi`) têm testes de unidade comuns, no próprio arquivo, em `#[cfg(test)] mod tests`. Para parsers, testes de **propriedade** com `proptest` cobrem muito mais que exemplos escolhidos à mão:

```
proptest! {
    #[test]
    fn cursor_faz_ida_e_volta(sequencia in 0i64..i64::MAX) {
        let cursor = codificar_cursor(sequencia);
        prop_assert_eq!(decodificar_cursor(Some(&cursor)).unwrap(),
            sequencia);
    }

    #[test]
    fn nosso_numero_nunca_entra_em_panic(texto in "\\PC{0,30}") {
        let _ = texto.parse::<NossoNumero>(); // qualquer texto: Ok ou Err,
        nunca_panic
    }
}
```

Quando uma propriedade falha, o `proptest` reduz a entrada ao menor contraexemplo e o guarda num arquivo de regressão, que passa a ser testado sempre. Para fuzzing guiado por cobertura, `cargo fuzz`.

10.2 Testes de rota com oneshot

```
use tower::ServiceExt; // oneshot

pub async fn corpo_json(resposta: Response) -> serde_json::Value {
    let bytes = axum::body::to_bytes(resposta.into_body(),
    usize::MAX).await.unwrap();
    serde_json::from_slice(&bytes).unwrap()
}

#[sqlx::test(migrations = "./migrations")]
async fn arquivo_de_outro_cliente_devolve_404(banco: PgPool) {
    let app = app_de_teste(banco.clone());
    let arquivo = semear_arquivo(&banco, CLIENTE_B).await;

    let resposta = app
        .oneshot(
            Request::get(format!("/v1/arquivos/{arquivo}"))
                .header(AUTHORIZATION, format!("Bearer {} ",
token_de_teste(CLIENTE_A, &["cobranca:ler"]))))
                .body(Body::empty())
                .unwrap(),
        )
        .await
        .unwrap();

    assert_eq!(resposta.status(), StatusCode::NOT_FOUND);
    assert_eq!(resposta.headers()[CONTENT_TYPE], "application/problem+json");
    assert_eq!(corpo_json(resposta).await["status"], 404);
}
```

`app_de_teste` chama o mesmo `montar_router` da produção, com um `AppState` de teste. Testar pelo router completo, e não chamando o handler direto, é o que pega erros de ordem de camadas, de extractor e de autenticação: os erros mais comuns numa API em axum.

10.3 Um banco real por teste

A macro `#[sqlx::test]` cria um banco de dados novo para cada teste, aplica as migrations e entrega um `PgPool` como parâmetro; no fim, o banco é apagado. Testes rodam em paralelo sem interferir uns nos outros e sem limpeza manual. Ela precisa de um

servidor PostgreSQL acessível por `DATABASE_URL`: o do `docker-compose` no desenvolvimento, e um serviço do pipeline (ou um container iniciado pelo `crate testcontainers`) no CI. A versão do PostgreSQL deve ser a mesma da produção, como nos outros cursos.

Dependências externas que não fazem parte do teste trocam de implementação no `AppState` de teste: o `Arc<dyn Storage>` (Módulo 5.5) recebe um storage em memória.

10.4 Autenticação real com uma chave de teste

Em vez de um validador falso, os testes usam o `Validador` de verdade configurado com uma chave RSA de teste, e geram tokens assinados com a chave privada correspondente:

```
// tests/comum/mod.rs
static CHAVE_PRIVADA: &[u8] = include_bytes!("chave-de-teste.pem"); // gerada
uma vez, só para testes

pub fn token_de_teste(cliente: Uuid, escopos: &[&str]) -> String {
    let claims = serde_json::json!({
        "sub": "teste", "cliente_id": cliente, "scope": escopos.join(" "),
        "iss": EMISSOR_DE_TESTE, "aud": AUDIENCIA_DE_TESTE,
        "exp": (Utc::now() + chrono::Duration::minutes(5)).timestamp(),
    });
    let mut cabecalho = jsonwebtoken::Header::new(Algorithm::RS256);
    cabecalho.kid = Some("teste".into());
    jsonwebtoken::encode(&cabecalho, &claims,
    &EncodingKey::from_rsa_pem(CHAVE_PRIVADA).unwrap()).unwrap()
}
```

O `Validador` ganha um construtor `com_chaves(JwkSet, emissor, audiencia)` que pula a descoberta pela rede. Com isso, o teste exercita a validação real: assinatura, emissor, audiência, expiração, extração do `cliente_id` e escopos. É o mesmo argumento do curso de .NET para usar as políticas reais com um esquema de autenticação de teste, levado um passo adiante: nem o esquema é falso. Um teste de token expirado é uma linha, com `exp` no passado.

10.5 Testes por comportamento

Os nomes descrevem o comportamento esperado, como nos outros cursos, e cada teste semeia os próprios dados num banco que é só dele:

```
#[sqlx::test] async fn
upload_sem_token_devolve_401_com_www_authenticate(banco: PgPool) { /* ... */
}
#[sqlx::test] async fn upload_sem_escopo_de_escrita_devolve_403(banco:
PgPool) { /* ... */ }
#[sqlx::test] async fn
mesmo_arquivo_duas_vezes_devolve_409_com_id_existente(banco: PgPool) { /* ...
*/ }
#[sqlx::test] async fn mesma_idempotency_key_devolve_a_mesma_resposta(banco:
PgPool) { /* ... */ }
#[sqlx::test] async fn listagem_com_cursor_nao_repete_nem_pula_itens(banco:
PgPool) { /* ... */ }
#[sqlx::test] async fn
exportacao_interrompida_nao_entrega_csv_truncado(banco: PgPool) { /* ... */ }
```

10.6 Contrato: as respostas batem com o OpenAPI

Este é o teste que fecha a lacuna do Módulo 7.3. Uma função auxiliar recebe o método, a rota e a resposta; procura a operação no documento gerado pelo `utoipa`; falha se o status não estiver declarado para aquela operação; e valida o corpo contra o schema declarado, convertendo o schema para JSON Schema e usando o crate `jjsonschema`. Chamada no fim de cada teste de rota, ela garante que tudo que a API realmente responde está no contrato, e que o contrato descreve o que a API realmente responde.

10.7 Snapshots

O crate `insta` guarda snapshots aprovados no repositório: o documento OpenAPI (Módulo 7.5) e os corpos de `ProblemDetails` de cada tipo de erro. Campos que mudam a cada execução, como `traceId`, são substituídos por *redactions* (`insta::assert_json_snapshot!(corpo, { ".traceId" => "[trace]" })`). Uma mudança no formato de erro aparece como diff a ser aprovado com `cargo insta review`.

10.8 Tempo e streams

Com a feature `test-util` do `tokio`, `#[tokio::test(start_paused = true)]` congela o relógio do runtime: o `interval` de um segundo do stream de progresso (Módulo 9.4) avança com `tokio::time::advance`, e um teste que verifica três eventos e o encerramento no estado final roda em milissegundos. Para o SSE, leia o corpo da resposta como stream (`into_body().into_data_stream()`) e confira os eventos um a um.

Exercício do Módulo 10

Monte o `app_de_teste` com `#[sqlx::test]`, storage em memória e o `Validador` com a chave de teste. Escreva os seis testes de 10.5. Implemente a função de validação de contrato e chame-a em todos eles; depois faça `obter` devolver um 403 não declarado e confirme que o teste de contrato falha, fechando o exercício do Módulo 7. Rode tudo com `cargo test` e meça quanto tempo a suíte leva com os bancos isolados em paralelo.

Módulo 11 — Operação: em produção e saudável

Operar a API significa as mesmas três coisas das outras duas: desligar sem derrubar requisições, dizer ao orquestrador se está saudável e permitir seguir uma requisição de ponta a ponta. Em Rust, o desligamento é composto à mão, como no Go, a partir de um sinal, uma flag de prontidão e um `CancellationToken`; a telemetria vem do `tracing`, com o `TraceLayer` do `tower-http` criando um span por requisição.

11.1 Desligamento gracioso

O `with_graceful_shutdown` do axum recebe uma future: quando ela termina, o servidor para de aceitar conexões e espera as requisições em andamento. A sequência de desligamento inteira cabe nessa future:

```
async fn sinal_de_desligamento(prontidao: Arc<AtomicBool>, desligando:
CancellationToken) {
    let mut term =
tokio::signal::unix::signal(SignalKind::terminate()).expect("handler de
SIGTERM");
    tokio::select! {
        _ = term.recv() => {}
        _ = tokio::signal::ctrl_c() => {}
    }
    tracing::info!("SIGTERM: saindo do balanceamento");
    prontidao.store(false, Ordering::Relaxed);           // 1. /health/ready
passa a responder 503
    tokio::time::sleep(Duration::from_secs(5)).await; // 2. tempo para o
Kubernetes tirar o pod dos endpoints
    desligando.cancel();                                // 3. encerra os
streams de SSE (Módulo 9.4)
}                                                         // 4. ao terminar: o
servidor para de aceitar e espera

// no main
let servidor = axum::serve(listener, app)
    .with_graceful_shutdown(sinal_de_desligamento(prontidao.clone(),
```

```

desligando.clone()))
    .into_future();

tokio::select! {
    resultado = servidor => resultado?,
    _ = async {
        desligando.cancelled().await;
        tokio::time::sleep(Duration::from_secs(25)).await; // prazo para as
requisições em andamento
    } => tracing::warn!("prazo de desligamento esgotado; saindo com
requisições em andamento"),
}

```

O passo 2 tem a mesma razão do curso de Go: sem ele, o processo para de aceitar conexões antes de o Kubernetes parar de enviá-las, e as requisições que chegam nesse intervalo falham. Fazer a espera dentro do processo dispensa o `preStop` com `sleep`, que não existiria numa imagem distroless. O `select!` final garante que o processo saia antes do `terminationGracePeriodSeconds` (35 segundos: 5 de espera + 25 de prazo + folga) mesmo que alguma requisição não termine.

11.2 Health checks

```

pub async fn vivo() -> StatusCode { StatusCode::OK } // só responde: o
processo está vivo

pub async fn pronto(State(estado): State<AppState>) -> StatusCode {
    if !estado.prontidao.load(Ordering::Relaxed) {
        return StatusCode::SERVICE_UNAVAILABLE; // desligando
    }
    let banco = tokio::time::timeout(Duration::from_secs(2),
sqlx::query("SELECT 1").execute(&estado.banco)).await;
    match banco {
        Ok(Ok(_)) => StatusCode::OK,
        _ => StatusCode::SERVICE_UNAVAILABLE,
    }
}

```

As regras de sempre: liveness nunca depende do banco, readiness depende só do essencial, e as duas rotas ficam fora de `/v1` (portanto fora da autenticação e do rate limiting).

11.3 Traces e logs

O `TraceLayer` cria um span por requisição; configurando `make_span_with`, o span ganha o nome e os atributos das convenções semânticas do OpenTelemetry e continua o trace de quem chamou:

```
TraceLayer::new_for_http()
    .make_span_with(|req: &Request| {
        let rota = req.extensions().get::<MatchedPath>
        ().map(MatchedPath::as_str).unwrap_or("desconhecida");
        let span = tracing::info_span!(
            "requisicao",
            otel.name = format!("{}", {rota}", req.method()), // "GET
/v1/arquivos/{id}"
            http.request.method = %req.method(),
            http.route = rota,
            http.response.status_code = tracing::field::Empty, // preenchido
na resposta
        );
        let pai = opentelemetry::global::get_text_map_propagator(|p|
p.extract(&HeaderExtractor(req.headers())));
        span.set_parent(pai); // continua o trace do traceparent recebido
span
    })
    .on_response(|resposta: &Response, _latencia: Duration, span: &Span| {
        span.record("http.response.status_code", resposta.status().as_u16());
    })
```

O `HeaderExtractor` (do crate `opentelemetry-http`) adapta os cabeçalhos ao propagador. A saída dos logs e a ponte para o OpenTelemetry são as do curso de arquivos em Rust (Módulo 9.2): `tracing-subscriber` em JSON e `tracing-opentelemetry` enviando os mesmos spans por OTLP, com `trace_id` idêntico nos logs e nos traces. Para chamadas de saída, o crate `request-tracing` cria spans de cliente e injeta o `traceparent`; para o banco, os spans das consultas do `sqlx` entram como filhos do span da requisição.

O `traceId` do `ProblemDetails` (Módulo 4.3) vem deste span: o cliente que reporta um erro informa o ID, e o trace inteiro aparece no backend de observabilidade.

11.4 Métricas

Métrica	Fonte	Para que serve
<code>http.server.request.duration</code> por <code>http.route</code> e status	Camada própria ou <code>tower-otel</code> sobre <code>MatchedPath</code>	p95/p99 por rota: a métrica do SLA
Requisições em andamento	A mesma camada	Saturação
Conexões do pool em uso e ociosas	<code>pool.size()</code> e <code>pool.num_idle()</code> do <code>sqlx</code> , como gauges	Pool pequeno ou banco lento
Tarefas vivas e fila do runtime	Métricas do tokio	Vazamento de tarefas, runtime sobrecarregado
Respostas 429 por cliente	O middleware de rate limiting	Cliente abusivo ou limite mal calibrado

A regra de cardinalidade é a de sempre: a rota vem do `MatchedPath` (o padrão com `{id}`), nunca da URL; o cliente só em métricas específicas e com cardinalidade conhecida.

11.5 Deploy

A imagem segue o modelo do curso de arquivos em Rust (Módulo 10.4): binário estático com musl e `mimalloc`, imagem *distroless static*, usuário não-root, com um detalhe: o build precisa de `SQLX_OFFLINE=true` para usar o cache `.sqlx/` comitado (Módulo 6.1) em vez de procurar um banco. No Deployment:

```
spec:
  template:
    spec:
      terminationGracePeriodSeconds: 35
      containers:
        - name: api
          image: <registro>/cobranca-api-rust:1.0.0
          ports: [{ containerPort: 8080 }]
          resources:
            requests: { cpu: "500m", memory: 64Mi }
            limits: { cpu: "2", memory: 256Mi }
          readinessProbe:
            httpGet: { path: /health/ready, port: 8080 }
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /health/live, port: 8080 }
            periodSeconds: 10
            failureThreshold: 3
```

O limite de CPU define quantas threads de trabalho o tokio cria (ou use `TOKIO_WORKER_THREADS`), como no curso de arquivos. O limite de memória cobre buffers de uploads em andamento (uma parte de 8 MiB por upload, Módulo 6.5) e o cache de títulos; meça no teste de carga antes de fixar.

11.6 Checklist de produção

- ☐ `Config::do_ambiente` falha na subida se faltar configuração
- ☐ Timeouts por grupo de rotas, `statement_timeout` no banco e `timeout` no cliente HTTP
- ☐ Desligamento com prontidão, espera, cancelamento dos streams e prazo final
- ☐ Health checks separados, fora de `/v1`
- ☐ `TraceLayer` com rota e status, logs JSON com `trace_id`, OTLP configurado
- ☐ Rate limiting por cliente e limite de uploads simultâneos
- ☐ Documento OpenAPI com snapshot e `oasdiff` no CI
- ☐ `cargo clippy -- -D warnings`, `cargo audit` e `cargo sqlx prepare --check` no CI
- ☐ Perfil de release com `panic = "unwind"`

Exercício do Módulo 11

Rode a API num cluster local com o Deployment acima. Gere carga constante com `oha` e faça um rollout: confirme que nenhuma requisição falha. Repita sem a espera de 5 segundos e conte os erros. Abra um stream de progresso e confirme que o rollout o

encerra. Envie uma requisição com `transparent` e encontre, no Aspire Dashboard, o trace com os spans da requisição e das consultas ao banco, e o mesmo `trace_id` no log JSON.

Projeto final — a API de cobrança em Rust, passo a passo

O mesmo projeto dos cursos de .NET e Go, agora com axum: a API que recebe arquivos de retorno, informa o status, consulta títulos, exporta ocorrências e transmite o progresso, sobre as tabelas compartilhadas com os workers e com o mesmo contrato de erros. Ao final, um cliente da API em .NET ou em Go deve conseguir trocar de URL sem mudar uma linha.

Estrutura

```
cobranca-api/
├─ Cargo.toml                axum, tokio, tower-http, serde, sqlx,
                               utoipa, jsonwebtoken, governor, moka...
├─ Cargo.lock
├─ .sqlx/                    cache de consultas para builds sem banco
├─ src/
│   └─ main.rs               configuração, telemetria, serve,
desligamento
│   └─ app.rs                AppState, montar_estado, montar_router,
rotas_v1
│   └─ erros.rs  extract.rs  auth.rs  limites.rs  dominio.rs  saude.rs
telemetria.rs
│   └─ arquivos/  titulos/  ocorrencias/
│       └─ armazenamento/          trait Storage, S3 e implementação em memória
├─ tests/
│   └─ comum/                  app_de_teste, token_de_teste, validação de
contrato, chave-de-teste.pem
│   └─ arquivos.rs  titulos.rs  ocorrencias.rs  seguranca.rs  contrato.rs
│       └─ snapshots/            snapshots do insta
└─ deploy/                    Dockerfile, k8s
```

As migrations ficam com o dono do schema (os workers ou um repositório próprio); a API as aplica só nos testes, pelo `#[sqlx::test(migrations = ...)]`, apontando para o mesmo diretório.

O AppState e o main

```
#[derive(Clone)]
pub struct AppState {
    pub banco: PgPool,
    pub jwt: Arc<auth::Validador>,
    pub storage: Arc<dyn armazenamento::Storage>,
    pub limites: Arc<limites::PorCliente>,
    pub uploads: Arc<limites::UploadsSimultaneos>,
    pub titulos: titulos::CacheDeTitulos, // moka: barato de clonar
    pub prontidao: Arc<AtomicBool>,
    pub desligando: CancellationToken,
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let config = Config::do_ambiente()?;
    let _telemetria = telemetria::iniciar(&config)?; // guarda: descarrega
    spans e métricas no Drop

    let estado = app::montar_estado(&config).await?;
    let (prontidao, desligando) = (estado.prontidao.clone(),
estado.desligando.clone());
    tokio::spawn(limites::limpar_periodicamente(estado.limites.clone(),
desligando.clone()));

    let app = app::montar_router(estado);
    let listener = tokio::net::TcpListener::bind(config.endereco).await?;
    tracing::info!(endereco = %config.endereco, "API iniciada");

    let servidor = axum::serve(listener, app)
        .with_graceful_shutdown(sinal_de_desligamento(prontidao,
desligando.clone()))
        .into_future();
    tokio::select! {
        r = servidor => r?,
        _ = prazo_de_desligamento(&desligando, Duration::from_secs(25)) => {
            tracing::warn!("prazo de desligamento esgotado");
        }
    }
    Ok(())
}
```

O router completo

```
pub fn montar_router(estados: AppState) -> Router {
    let (v1, documento) = rotas_v1(estados.clone()).split_for_parts(); //
    Router + OpenAPI (Módulo 7)

    Router::new()
        .nest("/v1", v1)
        .route("/openapi/v1.json", get(move || async move { Json(documento)
    })))

    .route("/health/live", get(saude::vivo))
    .route("/health/ready", get(saude::pronto))
    .fallback(erros::rota_inexistente)
    .method_not_allowed_fallback(erros::metodo_nao_permitido)
    .layer(
        ServiceBuilder::new()
            .layer(CatchPanicLayer::custom(erros::responder_panico))
            .layer(telemetria::camada_de_trace()) // span por
requisição com rota e traceparent
            .layer(middleware::from_fn(erros::registrar_instancia)) //
método e caminho para o ProblemDetails
            .layer(seguranca::cors())
            .layer(CompressionLayer::new()),
    )
    .with_state(estados)
}

fn rotas_v1(estados: AppState) -> OpenApiRouter<AppState> {
    let json = OpenApiRouter::new()
        .merge(arquivos::rotas_json())
        .merge(titulos::rotas())
        .layer(TimeoutLayer::new(Duration::from_secs(10)))
        .layer(DefaultBodyLimit::max(64 * 1024));

    let longas = arquivos::rotas_longas() // upload (com o middleware de
idempotência) e progresso
        .merge(ocorrencias::rotas_exportacao())
        .layer(DefaultBodyLimit::disable())
        .layer(RequestBodyLimitLayer::new(500 * 1024 * 1024))
        .layer(RequestBodyTimeoutLayer::new(Duration::from_secs(30))); //
corpo parado por 30 s: encerra

    json.merge(longas)
        .route_layer(middleware::from_fn_with_state(estados.clone(),
```

```

limites::por_cliente))
    .route_layer(middleware::from_fn_with_state(estado,
auth::autenticar)) // a mais externa: roda primeiro
}

```

Cada linha corresponde a uma decisão de algum módulo, e a ordem das camadas está comentada onde importa. As rotas de saúde e o documento OpenAPI ficam fora de `/v1`; tudo dentro de `/v1` é autenticado por construção.

As dez etapas

- ☐ **1. Fundação.** Projeto, `docker-compose` com PostgreSQL e Keycloak, `Config::do_ambiente`, `main` com o servidor e as rotas de saúde. Critério: `sobe`, `/health/ready` responde e configuração faltando impede a subida.
- ☐ **2. Erros.** `ProblemDetails`, `ErroDaApi`, `extractors Json / Path / Query` próprios, `fallback` e `CatchPanicLayer`. Critério: 400, 404, 405 e 500 saem todos como `application/problem+json` com `traceId`.
- ☐ **3. Consultas.** `GET /v1/titulos/{nosso_numero}` e `GET /v1/arquivos/{id}` com `sqlx` e `ArquivoDoCliente`. Critério: consultas verificadas em compilação; arquivo de outro cliente devolve 404.
- ☐ **4. Listagem.** `GET /v1/arquivos` com filtros validados e cursor keyset. Critério: percorrer 10 mil arquivos não repete nem pula itens; `EXPLAIN` sem `OFFSET`.
- ☐ **5. Upload.** `POST /v1/arquivos` em streaming, com hash, multipart no S3, duplicidade, outbox e idempotência. Critério: 400 MB com memória estável; mesmo arquivo duas vezes devolve 409; mesma `Idempotency-Key` devolve a mesma resposta.
- ☐ **6. Exportação e progresso.** CSV em streaming e SSE com `KeepAlive`. Critério: exportação interrompida não entrega CSV truncado; o stream termina no estado final e no desligamento.
- ☐ **7. Contrato.** `utoipa` com `routes!`, documento em `/openapi/v1.json`, `Scalar`, `snapshot` e `oasdiff`. Critério: renomear um campo aparece no diff do PR.
- ☐ **8. Segurança.** `Validador` com `JWKS`, `Usuario`, `ComEscopo`, `rate limiting` e uploads simultâneos. Critério: 401, 403, 404 e 429 com os cabeçalhos certos; rotação de chave sem reinício.
- ☐ **9. Testes.** `#[sqlx::test]`, `oneshot`, tokens de teste, validação de contrato em todo teste de rota, `proptest` nos parsers. Critério: suíte verde em paralelo; um status não declarado no OpenAPI quebra o teste.
- ☐ **10. Operação.** Telemetria OTLP, métricas por rota, desligamento gracioso, imagem estática e Deployment. Critério: rollout sob carga sem erros; um trace de ponta a ponta com o mesmo `trace_id` do log.

Crerios de aceite finais

- ☐ Toda resposta de erro ProblemDetails, idntica em formato as das APIs em .NET e Go
- ☐ Acrescentar uma variante a `ErroDaApi` sem decidir sua resposta no compila
- ☐ Nenhuma rota de negcio pode ser chamada sem autenticao, e nenhuma consulta de arquivo sem `ArquivoDoCliente`
- ☐ Consultas verificadas contra o schema no build (`cargo sqlx prepare --check` no CI)
- ☐ p95 abaixo de 200 ms nas consultas sob a carga de referencia
- ☐ Upload de 500 MB com memoria limitada a poucas partes de 8 MiB
- ☐ O documento OpenAPI descreve exatamente o que a API responde, provado pelos testes de contrato
- ☐ Rollout sob carga sem nenhuma requisicao falhando
- ☐ `clippy`, `cargo audit` e a suite de testes limpos no CI

Referncias e proximos passos

Com este curso, a serie fecha o quadro: a API e o pipeline de arquivos do sistema de cobranca existem em .NET, Go e Rust, sobre o mesmo banco, as mesmas mensagens e o mesmo contrato. O exercicio mais valioso agora e comparativo: rodar as tres APIs atrs do mesmo teste de carga e da mesma suite de testes de contrato, e observar no s latncia e memoria, mas quantos dos erros que cada modulo discutiu foram pegos pelo compilador, por testes ou s em producao. Proximos passos naturais em Rust: um cliente tipado da API gerado do documento OpenAPI (com o crate `progenitor`) e um endpoint gRPC com `tonic`, que tambem e `tower` e reaproveita as mesmas camadas.

Os outros cursos da serie

- [Curso: Minimal APIs no ASP.NET Core 10](#)
- [Curso: APIs HTTP em Go](#)
- [Curso: Rust e System Design para processamento de arquivos em alto volume](#)
- [Curso: Go e System Design para processamento de arquivos em alto volume](#)
- [Curso: .NET 10 e System Design para processamento de arquivos em alto volume](#)

Documentacao por modulo

Mdulo	Leitura
2 e 3. axum e tower	Documentacao do axum · Exemplos oficiais do axum · tower-http

Módulo	Leitura
3 e 4. serde e validação	serde · garde · thiserror
6. Dados	sqlx · async-stream
7. Contrato	utoipa · utoipa-axum · RFC 9457: Problem Details
8. Segurança	jsonwebtoken · governor · OWASP API Security Top 10
9. Performance	moka · SSE no axum
10. Testes	sqlx::test · proptest · insta
11. Operação	tracing · OpenTelemetry para Rust

Livros

- *Zero To Production In Rust*, Luca Palmieri: uma API completa em Rust com testes de integração, banco, telemetria e deploy; a leitura mais próxima deste curso.
- *Rust for Rustaceans*, Jon Gjengset: traits, tipos e `async` no nível em que extractors e camadas `tower` deixam de parecer mágica.
- *Designing Web APIs* (Brenda Jin, Saurabh Sahni e Amir Shevat) e *API Design Patterns* (JJ Geewax): desenho de APIs independente de linguagem, comuns aos três cursos.

Fontes sobre versões

- [Releases do axum](#): axum 0.8.9, de abril de 2026, é a versão atual; a sintaxe `{id}` de curingas vale desde o 0.8, e a rejeição de corpos JSON com caracteres sobrando desde o 0.8.5.
- [Anúncios de versão do Rust](#): Rust 1.99.0, de 1º de outubro de 2026, é a versão estável atual.